



天数智芯
Iluvatar CoreX

天数智芯

IxStream 流分析工具包使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150| 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 IxStream 流分析工具包使用指南	8
1.1 修订记录	8
1.2 概述	10
1.3 IxStream 功能模块	10
1.4 IxStream 快速上手	11
1.4.1 安装和配置 IxStream	11
1.4.1.1 支持的平台和操作系统	11
1.4.1.2 开始之前	11
1.4.1.3 安装 IxStream	12
1.4.1.4 设置环境变量	12
1.4.1.5 检查环境	13
1.4.2 基于 gst-launch-1.0 构建的简单的 Pipeline	13
1.4.2.1 配置 Plugin 属性	13
1.4.2.2 运行 Pipeline	14
1.4.3 运行示例程序	15
1.5 IxStream 示例程序	15
1.5.1 Python 示例程序	15
1.5.2 C++ 示例程序	16
1.6 IxStream Plugin 开发指南	17
1.6.1 IxStream SDK 中的元数据	18
1.6.1.1 如何获取元数据	18
1.6.1.2 如何添加用户自定义元数据	19
1.6.1.3 IxBatchMeta	21
1.6.1.4 IxFrameMeta	21
1.6.1.5 IxObjectMeta	22
1.6.1.6 IxClassifierMeta	22
1.6.1.7 IxLabelInfoList	22
1.6.1.8 IxDisplayMeta	22
1.6.2 Gst-IxVpuDecode	23
1.6.3 Gst-IxJpuDecode	24
1.6.4 Gst-IxJpuEncode	25
1.6.5 Gst-IxH264Encode	25
1.6.6 Gst-IxConverter	26
1.6.7 Gst-IxStreamMux	27
1.6.8 Gst-IxStreamDemux	28
1.6.9 Gst-IxPreprocess	29
1.6.10 Gst-IxInfer	32
1.6.11 Gst-IxTracker	36
1.6.12 Gst-IxOsd	38
1.6.13 Gst-IxMultiStreamTiler	39
1.6.14 Gst-IxAnalytics	40
1.6.15 Gst-IxSegVisual	42

1.7 IxStream Python API 参考	43
1.7.1 数据结构	43
1.7.1.1 pyixstream.IxBufSurfaceColorFormat	43
1.7.1.2 pyixstream.IxBufSurfaceMemType	44
1.7.1.3 pyixstream.IxBufSurfaceParams	45
1.7.1.4 pyixstream.IxBufSurface	45
1.7.1.5 pyixstream.IxMetaType	46
1.7.1.6 pyixstream.IxOSD_Mode	47
1.7.1.7 pyixstream.IxOSD_Arrow_Head_Direction	47
1.7.1.8 pyixstream.IxOSD_ColorParams	48
1.7.1.9 pyixstream.IxOSD_FontParams	48
1.7.1.10 pyixstream.IxOSD_TextParams	49
1.7.1.11 pyixstream.IxOSD_RectParams	49
1.7.1.12 pyixstream.IxOSD_LineParams	50
1.7.1.13 pyixstream.IxOSD_ArrowParams	50
1.7.1.14 pyixstream.IxOSD_CircleParams	51
1.7.1.15 pyixstream.IxBaseMeta	52
1.7.1.16 pyixstream.IxUserMeta	52
1.7.1.17 pyixstream.IxLabelInfo	52
1.7.1.18 pyixstream.IxClassifierMeta	53
1.7.1.19 pyixstream.IxDisplayMeta	53
1.7.1.20 pyixstream.IxObjectMeta	54
1.7.1.21 pyixstream.IxFrameMeta	55
1.7.1.22 pyixstream.IxBatchMeta	56
1.7.1.23 pyixstream.IxInferSegmentationMeta	56
1.7.1.24 pyixstream.IxAnalyticsObjInfo	57
1.7.1.25 pyixstream.IxAnalyticsFrameMeta	57
1.7.1.26 pyixstream.IxInferDims	58
1.7.1.27 pyixstream.IxInferDimsCHW	58
1.7.1.28 pyixstream.IxInferDataType	59
1.7.1.29 pyixstream.IxInferLayerInfo	59
1.7.1.30 pyixstream.IxInferNetworkInfo	60
1.7.1.31 pyixstream.IxInferObjectDetectionInfo	60
1.7.2 函数接口	61
1.7.2.1 pyixstream.ix_acquire_frame_meta_from_pool	61
1.7.2.2 pyixstream.ix_add_frame_meta_to_batch	61
1.7.2.3 pyixstream.ix_remove_frame_meta_from_batch	62
1.7.2.4 pyixstream.ix_acquire_obj_meta_from_pool	62
1.7.2.5 pyixstream.ix_add_obj_meta_to_frame	62
1.7.2.6 pyixstream.ix_remove_obj_meta_from_frame	63
1.7.2.7 pyixstream.ix_acquire_classifier_meta_from_pool	63
1.7.2.8 pyixstream.ix_add_classifier_meta_to_obj	64
1.7.2.9 pyixstream.ix_remove_classifier_meta_from_obj	64
1.7.2.10 pyixstream.ix_acquire_display_meta_from_pool	64
1.7.2.11 pyixstream.ix_add_display_meta_to_frame	65
1.7.2.12 pyixstream.ix_remove_display_meta_from_frame	65
1.7.2.13 pyixstream.ix_acquire_label_info_meta_from_pool	65

1.7.2.14	pyixstream.ix_add_label_info_meta_to_classifier	66
1.7.2.15	pyixstream.ix_remove_label_info_meta_from_classifier	66
1.7.2.16	pyixstream.ix_acquire_user_meta_from_pool	67
1.7.2.17	pyixstream.ix_add_user_meta_to_batch	67
1.7.2.18	pyixstream.ix_remove_user_meta_from_batch	67
1.7.2.19	pyixstream.ix_add_user_meta_to_frame	68
1.7.2.20	pyixstream.ix_remove_user_meta_from_frame	68
1.7.2.21	pyixstream.ix_add_user_meta_to_obj	69
1.7.2.22	pyixstream.ix_remove_user_meta_from_obj	69
1.7.2.23	pyixstream.ix_get_nth_frame_meta	69
1.7.2.24	pyixstream.ix_clear_frame_meta_list	70
1.7.2.25	pyixstream.ix_clear_obj_meta_list	70
1.7.2.26	pyixstream.ix_clear_classifier_meta_list	71
1.7.2.27	pyixstream.ix_clear_label_info_meta_list	71
1.7.2.28	pyixstream.ix_clear_display_meta_list	71
1.7.2.29	pyixstream.ix_clear_batch_user_meta_list	72
1.7.2.30	pyixstream.ix_clear_frame_user_meta_list	72
1.7.2.31	pyixstream.ix_get_user_meta_type	73
1.7.2.32	pyixstream.gst_buffer_add_ix_meta	73
1.7.2.33	pyixstream.gst_buffer_get_ix_meta	73
1.7.2.34	pyixstream.gst_buffer_get_ix_batch_meta	74
1.7.2.35	pyixstream.ix_acquire_meta_lock	74
1.7.2.36	pyixstream.ix_release_meta_lock	75
1.7.2.37	pyixstream.ix_create_batch_meta	75
1.7.2.38	pyixstream.ix_destory_batch_meta	75
1.7.2.39	pyixstream.get_ix_buf_surface	76
1.7.2.40	pyixstream.get_ix_buf_surface_gpu	76
1.7.2.41	pyixstream.get_segmentation_masks	76
1.7.2.42	pyixstream.get_ix_LayerInfo	77
1.7.2.43	pyixstream.user_copyfunc	77
1.7.2.44	pyixstream.user_releasefunc	77
1.7.2.45	pyixstream.alloc_buffer	78
1.7.2.46	pyixstream.free_buffer	78
1.7.2.47	pyixstream.free_gbuffer	78
1.7.2.48	pyixstream.get_string	79
1.7.2.49	pyixstream.get_ptr	79
1.8	IxStream C++ API 参考	80
1.8.1	数据结构	80
1.8.1.1	IxBufSurfaceColorFormat	80
1.8.1.2	IxBufSurfaceMemType	81
1.8.1.3	IxBufSurfacePlaneParams	81
1.8.1.4	IxBufSurfaceParams	82
1.8.1.5	IxBufSurface	82
1.8.1.6	IxMetaType	82
1.8.1.7	IxOSD_Arrow_Head_Direction	83
1.8.1.8	IxOSD_ColorParams	84
1.8.1.9	IxOSD_FontParams	84

1.8.1.10	IxOSD_TextParams	84
1.8.1.11	IxOSD_RectParams	85
1.8.1.12	IxOSD_MaskParams	85
1.8.1.13	IxOSD_LineParams	86
1.8.1.14	IxOSD_PolygonParams	86
1.8.1.15	IxOSD_ArrowParams	86
1.8.1.16	IxOSD_CircleParams	87
1.8.1.17	IxBaseMeta	87
1.8.1.18	IxUserMeta	88
1.8.1.19	IxLabelInfo	88
1.8.1.20	IxClassifierMeta	88
1.8.1.21	IxDisplayMeta	89
1.8.1.22	IxObjectMeta	89
1.8.1.23	IxFrameMeta	90
1.8.1.24	IxBatchMeta	91
1.8.1.25	IxInferSegmentationMeta	91
1.8.1.26	IxAnalyticsObjInfo	92
1.8.1.27	IxAnalyticsFrameMeta	92
1.8.1.28	IxInferDims	93
1.8.1.29	IxInferDimsCHW	93
1.8.1.30	IxInferDataType	93
1.8.1.31	IxInferLayerInfo	94
1.8.1.32	IxInferNetworkInfo	94
1.8.1.33	IxInferObjectDetectionInfo	94
1.8.1.34	IxInferAttribute	95
1.8.1.35	IxInferParseDetectionParams	95
1.8.2	函数接口	96
1.8.2.1	ix_acquire_frame_meta_from_pool	96
1.8.2.2	ix_add_frame_meta_to_batch	96
1.8.2.3	ix_remove_frame_meta_from_batch	96
1.8.2.4	ix_acquire_obj_meta_from_pool	97
1.8.2.5	ix_add_obj_meta_to_frame	97
1.8.2.6	ix_remove_obj_meta_from_frame	97
1.8.2.7	ix_acquire_classifier_meta_from_pool	98
1.8.2.8	ix_add_classifier_meta_to_obj	98
1.8.2.9	ix_remove_classifier_meta_from_obj	99
1.8.2.10	ix_acquire_display_meta_from_pool	99
1.8.2.11	ix_add_display_meta_to_frame	99
1.8.2.12	ix_remove_display_meta_from_frame	100
1.8.2.13	ix_acquire_label_info_meta_from_pool	100
1.8.2.14	ix_add_label_info_meta_to_classifier	100
1.8.2.15	ix_remove_label_info_meta_from_classifier	101
1.8.2.16	ix_acquire_user_meta_from_pool	101
1.8.2.17	ix_add_user_meta_to_batch	101
1.8.2.18	ix_remove_user_meta_from_batch	102
1.8.2.19	ix_add_user_meta_to_frame	102
1.8.2.20	ix_remove_user_meta_from_frame	103

1.8.2.21 ix_add_user_meta_to_obj	103
1.8.2.22 ix_remove_user_meta_from_obj	103
1.8.2.23 ix_get_nth_frame_meta	104
1.8.2.24 ix_clear_frame_meta_list	104
1.8.2.25 ix_clear_obj_meta_list	104
1.8.2.26 ix_clear_classifier_meta_list	105
1.8.2.27 ix_clear_label_info_meta_list	105
1.8.2.28 ix_clear_display_meta_list	105
1.8.2.29 ix_clear_batch_user_meta_list	106
1.8.2.30 ix_clear_frame_user_meta_list	106
1.8.2.31 ix_get_user_meta_type	106
1.8.2.32 gst_buffer_add_ix_meta	107
1.8.2.33 gst_buffer_get_ix_meta	107
1.8.2.34 gst_buffer_get_ix_batch_meta	108
1.8.2.35 IxInferParseCustomFunc	108
1.8.2.36 IxInferClassifierParseCustomFunc	108
1.8.2.37 IxInferSemSegmentationParseCustomFunc	109
1.8.2.38 ix_parse_file_source	109
1.8.2.39 ix_parse_rtsp_source	110
1.8.2.40 ix_parse_vpu_decode	110
1.8.2.41 ix_parse_streammux	111
1.8.2.42 ix_parse_tracker	111
1.8.2.43 ix_parse_osd	112
1.8.2.44 ix_parse_tiler	112
1.8.2.45 ix_parse_gie	113
1.8.2.46 ix_parse_file_sink	113
1.8.2.47 ix_parse_fake_sink	114
1.8.2.48 ix_parse_h264_encode	114
1.8.2.49 ix_parse_jpu_encode	115
1.8.2.50 ix_parse_analytics	115
1.8.2.51 ix_parse_seg_visual	116
1.8.2.52 ix_parse_converter	116
1.8.2.53 ix_parse_preprocess	117
1.8.2.54 ix_get_json_list	117
1.8.2.55 ix_parse_file_source_list	118
1.8.2.56 ix_parse_jpu_encode_list	118
1.8.2.57 ix_parse_file_sink_list	118
1.8.2.58 ix_parse_h264_encode_list	119
1.8.2.59 ix_parse_rtsp_source_list	119
1.8.2.60 ix_parse_jpu_decode	120
1.8.2.61 ix_parse_multi_file_source	120
1.9 常见问题	121
1.9.1 环境相关问题	121
1.10 IxStream 与 DeepStream 配置接口对比一览表	123

1 IxStream 流分析工具包使用指南

1.1 修订记录

- COREX01-MR430-UG11-00: 2025/6/30
 - 更新本次发布的 IxStream 版本为 v1.2.7
 - 更新**支持的平台和操作系统**，更新天数智算软件栈版本
 - 更新**IxStream 示例程序**，增加示例程序获取及默认安装路径说明
 - 更新**Python 示例程序**:
 - * 更新 ixstream_sample1 说明描述
 - * 更新 ixstream_sample_infer_yolo 说明描述
 - 更新**C++ 示例程序**:
 - * 更新 ixstream_sample1 说明描述
 - * 更新 ixstream_sample_infer_yolo 说明描述
 - 更新**IxStream SDK 中的元数据**:
 - * 更新 IxFrameMeta: 增加 buf_pts 和 source_id 参数
 - * 更新 IxObjectMeta: 增加 unique_component_id 和 object_id 参数
 - * 更新 IxClassifierMeta: 增加 unique_component_id 参数
 - 更新**Gst-IxVpuDecode**:
 - * 增加功能: 支持低延迟模式, NV12 格式输出
 - * 增加属性 low-latency-mode 和 output-format
 - * 更新**Gst-IxConverter**:
 - * 更新输入输出及功能: 支持 NV21 格式, BGR format
 - * 增加 drop-only 属性说明
- COREX01-MR420-UG11-01: 2025/4/14
 - 更新**Python 示例程序**，更新 ixstream_sample_tiler 示例程序名称
 - 更新**Gst-IxPreprocess**，更新支持的参数
 - 更新**Gst-IxInfer**:
 - * 更新支持的**网络推理模型列表**
 - * 更新**YOLO 后处理函数及说明** 中各模型的出处链接
 - * 补充支持的参数
 - 更新**Gst-IxAnalytics**，修正支持的参数
- COREX01-MR420-UG11-00: 2025/1/24
 - 更新本次发布的 IxStream 版本为 v1.2.2
 - 更新**安装 IxStream**:
 - * **设置环境变量**，新增确认 libtvm.so 和 libixrt.so 安装路径的操作
 - * **检查环境**，新增部分 IxStream Plugin 的检查
 - 本次发布将 IxStream 示例程序打包进安装包中，无需再联系应用工程师获取，IxStream 示例程序默认安装路径为 /opt/ixstream_samples
 - 新增以下**Python 示例程序**:

- * ixstream_sample_segmentation
- 新增以下C++ 示例程序：
 - * ixstream_sample5
 - * ixstream_sample_image_decode
 - * ixstream_sample_postprocess
- 更新Gst-IxInfer，新增支持 YOLOv9 和 YOLOv10 模型
- 新增IxStream Python API 参考
- 新增IxStream C++ API 参考
- COREX01-MR412-UG11-00：2024/9/2
 - 更新本次发布的 IxStream 版本为 v1.1.4
 - 安装 IxStream
 - * 更新获取 IxStream 安装包的方式，提示用户可在 [天数智芯官网](#) 下载
 - * 更新 whl 包名称
 - 安装和配置 IxStream
 - * 支持的平台和操作系统 更新软件版本
 - * 开始之前 更新安装 GStreamer 相关依赖和安装 PyGObject 相关命令
 - 更新Python 示例程序 和C++ 示例程序
 - IxStream Plugin 开发指南
 - * 更新以下 Plugin 信息：
 - Gst-IxJpuDecode
 - Gst-IxH264Encode
 - Gst-IxConverter
 - Gst-IxInfer
 - Gst-IxTracker
 - Gst-IxOsd
 - * 新增以下 Plugin 支持：
 - Gst-IxJpuDecode
 - Gst-IxPreprocess
 - Gst-IxAnalytics
 - Gst-IxSegVisual
 - 常见问题 新增“问题 4：jpegparse 解析报错”和“问题 5：qtdemux 在某些视频上报错”
 - 更新IxStream 与 DeepStream 配置接口对比一览表
- COREX01-MR400-UG11-01：2024/5/16
 - 更新Python 示例程序：
 - * 增加以下示例：
 - ixstream_sample1_osd
 - ixstream_sample1_usbcam
 - ixstream_sample2
 - * 删除示例 ixstream_tiler_sample
 - 增加C++ 示例程序
- COREX01-MR400-UG11-00：2024/4/3

本次文档发布内容与 V3.2.1-MR 文档相比 (COREX01-MR321-UG11-00)，有以下更新：

- 更新本次发布的 IxStream 版本为 v1.0.5
- 更新兼容的天数智算软件栈的版本为 v4.0.0
- 更新**安装和配置 IxStream**小节中的 IxStream 所需依赖
- 新增**基于 gst-launch-1.0 构建的简单的 Pipeline**小节，提供一个 Pipeline 供用户快速上手 IxStream
- 新增**IxStream 示例程序**章节
- 更新**如何获取元数据**小节，更新头文件
- 更新 IxFrameMeta 元数据中关于“num_obj_meta”上限的说明
- 新增**Gst-IxMultiStreamTiler** Plugin
- 更新各 Plugin 属性
- 更新**常见问题**章节

1.2 概述

IxStream (Iluvatar Stream) SDK 是天数智芯提供的面向 AI 智能视频分析 (IVA) 以及多传感器数据流处理的工具包，旨在支持用户能够在部署了天数智芯加速卡和软件栈的硬件平台上快速开发和部署实时数据流分析 Pipeline。当前提供的版本是 v1.2.7。

开发者只需要专注于构建核心的神经网络模型任务，无需从头开始设计端到端的数据流处理 Pipeline。一个常见的 IVA 工作流程如下图所示：

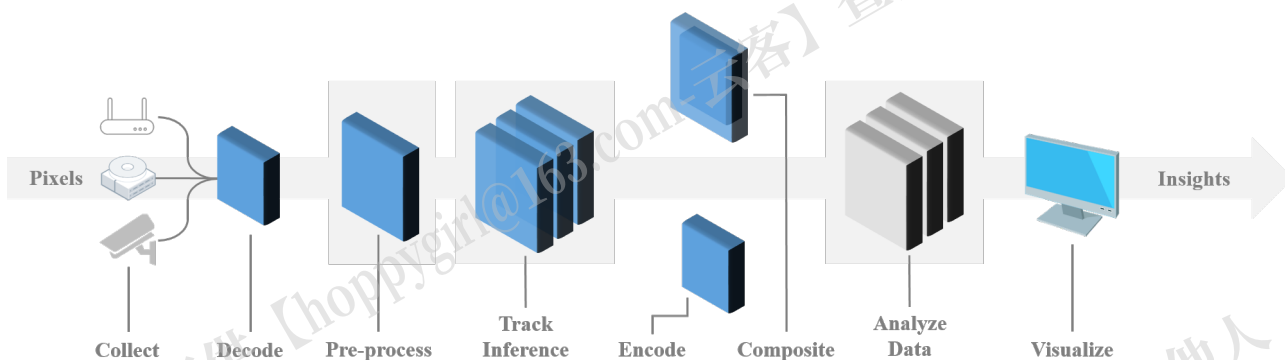


图 1: IVA 工作流程

1.3 IxStream 功能模块

IxStream 是基于 GStreamer 框架实现的，GStreamer 是一个用于开发流媒体应用的跨平台开源框架，采用了 Plugin 和 Pipeline 的体系结构。GStreamer 框架中的所有功能模块都被实现成可以插拔的 Plugin，并且能够很方便地插入到任意一个 Pipeline 中。

当前，IxStream 主要包含以下功能模块：

- CODEC 功能模块：基于 VPU/JPU 的视频/图片编解码方案
- Mux/Demux 功能模块：对多路输入输出做统一管理
- Infer 功能模块：基于 IGIE 推理框架/ixRT 推理引擎对输入的图像或对象进行推理
- Tracker 功能模块：跟踪检测到的具有持久 ID 的对象
- Osd 功能模块：对输出的图片进行绘制

IxStream 核心功能由多个硬件加速器 Plugin 组成，这些 Plugin 使用 GPU、VPU、JPU 等加速器。通过在专用加速器中执行所有计算密集型操作，IxStream 可以实现视频分析应用程序的最高性能。了解更多 Plugin 信息，请参考[IxStream Plugin 开发指南](#)。

IxStream 构建在天数智算软件栈的各种库之上，例如 CUDA、CODEC、CV-CUDA、IGIE 等。IxStream 在 Plugin 中对这些库进行了封装并提供了简单的属性配置接口，使开发人员可以轻松构建视频分析管道，无需单独学习这些库。

1.4 IxStream 快速上手

本小节旨在介绍安装和配置 IxStream 的操作步骤，并提供一个基于 `gst-launch-1.0` 构建的简单的 Pipeline。

1.4.1 安装和配置 IxStream

1.4.1.1 支持的平台和操作系统

下表提供了有关安装和配置 IxStream 兼容的平台和软件版本信息：

IxStream 版本	IxStream1.2.7
CPU 架构	x86 ARM
天数智芯加速卡	MR-V50 MR-V100 BI-V150 BI-V150S
操作系统	Ubuntu 18.04 Ubuntu 20.04 CentOS 7.9
天数智算软件栈	4.3.0
GStreamer	>=1.10.5

1.4.1.2 开始之前

1. 请确认已安装天数智算软件栈、IGIE 推理框架和 ixRT 推理引擎，详情请参考《软件栈安装指南》、《IGIE 推理框架使用指南》和《ixRT 推理引擎使用指南》。
2. 请确认已安装 GStreamer 相关依赖。
 - Ubuntu 环境请使用以下命令安装 GStreamer：

```
$ apt-get install glib2.0 libgstreamer1.0 libgstreamer1.0-dev gstreamer1.0-tools
↪ gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-plugins-bad
↪ gstreamer1.0-plugins-ugly libgstrtspserver-1.0 libopenblas-dev
```

- CentOS 环境请使用以下命令安装 GStreamer:

```
$ yum -y install glib2 gstreamer1 gstreamer1-devel gstreamer1-plugins-base
↪ gstreamer1-plugins-good gstreamer1-plugins-bad-free
↪ gstreamer1-plugins-ugly-free gstreamer1-rtsp-server-devel
```

3. 请确认已安装 PyGObject。

- Ubuntu 环境请使用以下命令安装 PyGObject:

```
$ apt-get install libgirepository1.0-dev libcairo2-dev
```

- CentOS 环境请使用以下命令安装 PyGObject:

```
$ yum install pygobject3-devel cairo-devel cairo-gobject-devel pkg-config
↪ python3-devel
```

1.4.1.3 安装 IxStream

天数智芯提供了 IxStream 的 C++ 和 Python 安装包 (Python 包需要依赖 C++ 包), 请在 [天数智芯官网](#) 的 **客户支持 > 资源中心** 页面进行下载。

通过以下命令安装 IxStream:

```
$ ./ixstream-{version}_corex-{v.r.m}_{ARCH}.run # C++ 安装
$ pip3 install pyixstream-{version}
↪ +corex.{v.r.m}-cp{python_version}-cp{python_version}-linux_{ARCH}.whl # Python 安装, 包
↪ 名如 pyixstream-1.1.4+corex.4.1.2-cp310-cp310-linux_x86_64.whl
```

1.4.1.4 设置环境变量

IxStream 安装完成后, 需要设置环境变量。以下命令以 Python v3.10 为例。

1. 确认 libtvm.so 和 libixrt.so 的安装路径:

```
$ find /usr/ -name libtvm.so
/usr/local/lib/python3.10/dist-packages/tvm/libtvm.so
/usr/local/lib/python3.10/dist-packages/tvm/cpp/libtvm.so
```

```
$ find /usr/ -name libixrt.so
/usr/local/lib/python3.10/dist-packages/tensorrt/lib/libixrt.so
/usr/local/corex/lib64/libixrt.so
```

2. 设置 GST_PLUGIN_PATH 和 LD_LIBRARY_PATH 环境变量 (LD_LIBRARY_PATH 需将 libtvm.so 和 libixrt.so 的路径包含在内):

```
$ export GST_PLUGIN_PATH=$GST_PLUGIN_PATH:/usr/local/corex/lib64/plugins
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/corex/lib64:/usr/local/lib/
↪ python3.10/dist-packages/tvm
```

1.4.1.5 检查环境

1. 以上环境配置完成后，您可以执行 **gst-inspect-1.0** 命令查看到 IxStream 相关的 Plugin 信息 (为防止缓存影响，执行命令前需要删除 GStreamer 的缓存)：

```
$ rm -rf /root/.cache/gstreamer-1.0/ # 删除缓存
$ gst-inspect-1.0 | grep Gst # 查看 Plugin 信息
```

2. 输出以下信息表示环境配置成功 (共 14 个 IxStream Plugin)：

```
ixstream_vpudecode: Gst-IxVpuDecode: Gst-IxVpuDecode plugin
ixstream_tracker: Gst-IxTracker: Gst-IxTracker plugin
ixstream_streammux: Gst-IxStreamMux: Gst-IxStreamMux plugin
ixstream_streamdemux: Gst-IxStreamDemux: Gst-IxStreamDemux plugin
ixstream_segvisual: Gst-IxSegVisual: Gst-IxSegVisual plugin
ixstream_preprocess: Gst-IxPreprocess: Gst-IxInfer plugin
ixstream_osd: Gst-IxOsd: Gst-IxOsd plugin
ixstream_tiler: Gst-IxMultiStreamTiler: Gst-IxMultiStreamTiler plugin
ixstream_jpuencode: Gst-IxJpuEncode: Gst-IxJpuEncode plugin
ixstream_jpudecode: Gst-IxJpuDecode: Gst-IxJpuDecode plugin
ixstream_infer: Gst-IxInfer: Gst-IxInfer plugin
ixstream_h264encode: Gst-IxH264Encode: Gst-IxH264Encode plugin
ixstream_converter: Gst-IxConverter: Gst-IxConverter plugin
ixstream_analytics: Gst-IxAnalytics: Gst-IxAnalytics plugin
```

如果输出信息与上述有不同，请参考[开始之前](#)和[设置环境变量](#)小节重新进行准备工作，并检查环境是否配置成功。

1.4.2 基于 gst-launch-1.0 构建的简单的 Pipeline

1.4.2.1 配置 Plugin 属性

在构建 Pipeline 之前，您需要熟悉各个 Plugin 的属性，以针对不同的需求进行差异化配置。您可以通过 **gst-inspect-1.0** 命令查看对应的 Plugin 的属性，如下图所示：

```
root@ :~# gst-inspect-1.0 Gst-IxVpuDecode
Factory Details:
Rank: none (0)
Long-name: Gst-IxVpuDecode plugin
Klass: Gst-IxVpuDecode functionality
Description: gstreamer vpu decode element
Author: Iluvatar CoreX

Plugin Details:
Name: ixstream_vpu decode
Description: Template ixvpudecode
Filename: /usr/local/corex/plugins/libixstream_vpu decode.so
Version: 1.0.0
License: LGPL
Source module: ixstream
Binary package: IxStream
Origin URL: http:// /ixstream.git

GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstBaseTransform
+----GstIxVpuDecode

Pad Templates:
SRC template: 'src'
Availability: Always
Capabilities: ANY

SINK template: 'sink'
Availability: Always
Capabilities: ANY

Element has no clocking capabilities.
Element has no URI handling capabilities.

Pads:
SINK: 'sink'
Pad Template: 'sink'
SRC: 'src'
Pad Template: 'src'

Element Properties:
drop-frame-interval : Drop frame
flags: readable, writable, changeable only in NULL or READY state
Unsigned Integer. Range: 1 - 30 Default: 1
frame-height : Height of each frame. This must be same as the input size.
flags: readable, writable, changeable only in NULL or READY state
Unsigned Integer. Range: 0 - 4294967295 Default: 1080
frame-width : Width of each frame. This must be same as the input size.
flags: readable, writable, changeable only in NULL or READY state
Unsigned Integer. Range: 0 - 4294967295 Default: 1920
gpu-id : Set GPU Device ID
flags: readable, writable, changeable only in NULL or READY state
Unsigned Integer. Range: 0 - 4294967295 Default: 0
name : The name of the object
flags: readable, writable
String. Default: "ixvpudecode0"
parent : The parent of the object
flags: readable, writable
Object of type "GstObject"
qos : Handle Quality-of-Service events
flags: readable, writable
Boolean. Default: false
video-format : video format of each frame.
flags: readable, writable, changeable only in NULL or READY state
Enum "Gst-IxVpuDecodeVideoFormat" Default: 0, "h264"
(0): h264 - H264 Format
(12): h265 - H265 Format
(14): avs2 - AVS2 Format
```

图 2: 查看对应 Plugin 属性

更多 IxStream Plugin 的属性说明，请参考IxStream Plugin 开发指南。

1.4.2.2 运行 Pipeline

以一个简单的编解码示例为例，运行以下命令即可运行 Pipeline 程序：

```
$ gst-launch-1.0 filesrc location="./sample_720p.h264" ! Gst-IxVpuDecode frame-width=1280
↪ frame-height=720 video-format=0 ! Gst-IxStreamMux live-source=false ! Gst-IxJpuEncode
↪ save-file-path="./jpeg_out" ! fakesink
```

1.4.3 运行示例程序

安装完 IxStream 后，您可以 `${path_to_ixstream}/ixstream_samples` 路径下找到 IxStream 的示例程序，默认安装路径为 `/opt/ixstream_samples`。

更多示例程序说明，请参考 [IxStream 示例程序](#)。

1.5 IxStream 示例程序

IxStream 安装包提供了 `ixstream_samples` 的示例程序，默认安装路径为 `/opt/ixstream_samples`。

1.5.1 Python 示例程序

`ixstream_samples` 中提供的 Python 示例程序简要说明如下表所示。

示例程序	所在仓库中的目录	说明
<code>ixstream_sample1</code>	<code>ixstream_samples/python/ixstream_sample1</code>	基于 yolov10 进行检测的简单示例，并对检测到的对象进行画框等操作；输入为 720P 的 h264 视频文件，输出为 jpeg 图片
<code>ixstream_sample1_osd</code>	<code>ixstream_samples/python/ixstream_sample1_osd</code>	基于 <code>ixstream_sample1</code> 增加了目标物体的统计功能，并将结果显示在图片上
<code>ixstream_sample1_rtsp_out</code>	<code>ixstream_samples/python/ixstream_sample1_rtsp_out</code>	基于 <code>ixstream_sample1</code> 将输出更改为 rtsp 流
<code>ixstream_sample1_usbcam</code>	<code>ixstream_samples/python/ixstream_sample1_usbcam</code>	基于 <code>ixstream_sample1</code> 将输入更改为摄像头输入
<code>ixstream_sample2</code>	<code>ixstream_samples/python/ixstream_sample2</code>	基于 <code>ixstream_sample1</code> 增加 tracker 和二级推理功能
<code>ixstream_sample3</code>	<code>ixstream_samples/python/ixstream_sample3</code>	基于 <code>ixstream_sample1</code> 增加多路输入/输出支持
<code>ixstream_sample4</code>	<code>ixstream_samples/python/ixstream_sample4</code>	基于 <code>ixstream_sample3</code> 将输出更改为输出到屏幕
<code>ixstream_sample5</code>	<code>ixstream_samples/python/ixstream_sample5</code>	基于 <code>ixstream_sample3</code> 支持更多的输入输出 (输入：h264 视频文件，mp4 视频文件，rtsp 流；输出：jpeg 图片，h264 视频文件)

示例程序	所在仓库中的目录	说明
ixstream_sample_analytics	ixstream_samples/python/ ixstream_sample_analytics	基于 ixstream_sample1 增加 tracker 和分析功能
ixstream_sample_custom_binding	ixstream_samples/python/ ixstream_sample_custom_binding	如何添加并访问用户自定义元数据
ixstream_sample_get_buffer	ixstream_samples/python/ ixstream_sample_get_buffer	如何获取 GST buffer 数据
ixstream_sample_infer_yolo	ixstream_samples/python/ ixstream_sample_infer_yolo	基于 ixstream_sample1 增加更多 yolo 模型的支持
ixstream_sample_pause_and_switch_input	ixstream_samples/python/ ixstream_sample_pause_and_switch_input	如何动态暂停和切换输入源
ixstream_sample_segmentation	ixstream_samples/python/ ixstream_sample_segmentation	使用 unet 分割模型对图片进行分割，并对分割结果进行可视化
ixstream_sample_tiler	ixstream_samples/python/ ixstream_sample_tiler	获取多路 rtsp 流，解码后将多路数据拼接成一路，最后将拼接后的图片保存为 jpeg 图片

1.5.2 C++ 示例程序

ixstream_samples 中提供的 C++ 示例程序简要说明如下表所示。

示例程序	所在仓库中的目录	说明
ixstream_sample1	ixstream_samples/C++/ ixstream_sample1	基于 yolov10 进行检测的简单示例，并对检测到的对象进行画框等操作；输入为 720P 的 h264 视频文件，输出为 jpeg 图片
ixstream_sample1_osd	ixstream_samples/C++/ ixstream_sample1_osd	基于 ixstream_sample1 增加了目标物体的统计功能，并将结果显示在图片上
ixstream_sample1_rtsp_out	ixstream_samples/C++/ ixstream_sample1_rtsp_out	基于 ixstream_sample1 将输出更改为 rtsp 流
ixstream_sample1_usbcam	ixstream_samples/C++/ ixstream_sample1_usbcam	基于 ixstream_sample1 将输入更改为摄像头输入
ixstream_sample2	ixstream_samples/C++/ ixstream_sample2	基于 ixstream_sample1 增加 tracker 和二级推理功能

示例程序	所在仓库中的目录	说明
ixstream_sample3	ixstream_samples/C++/ ixstream_sample3	基于 ixstream_sample1 增加多路输入/输出支持
ixstream_sample4	ixstream_samples/C++/ ixstream_sample4	基于 ixstream_sample3 将输出更改为输出到屏幕
ixstream_sample5	ixstream_samples/C++/ ixstream_sample5	基于 ixstream_sample3 支持更多的输入输出 (输入: h264 视频文件, mp4 视频文件, rtsp 流; 输出: jpeg 图片, h264 视频文件)
ixstream_sample_image_decode	ixstream_samples/C++/ ixstream_sample_image_decode	基于 ixstream_sample1 将输入更改为 jpeg 图片和 mjpeg 格式视频文件
ixstream_sample_infer_yolo	ixstream_samples/C++/ ixstream_sample_infer_yolo	基于 ixstream_sample1 增加更多 yolo 模型的支持
ixstream_sample_pause_and_switch_input	ixstream_samples/C++/ixstream_sample_pause_and_switch_input	如何动态暂停和切换输入源
ixstream_sample_postprocess	ixstream_samples/C++/ ixstream_sample_postprocess	以 YOLOv4 模型为例, 如何编写自定义模型后处理程序

1.6 IxStream Plugin 开发指南

本小节旨在为开发人员介绍 IxStream 支持的 Plugin, 并提供这些 Plugin 的输入和输出等信息, 另外, 在本小节中, 您还可以了解 IxStream SDK 中使用的元数据。

下表列出了当前 IxStream SDK 中提供的 Plugin:

Plugin	说明
Gst-IxVpuDecode	基于 VPU 硬件加速的视频解码插件
Gst-IxJpuDecode	基于 JPU 硬件加速的 JPEG 图片解码插件
Gst-IxJpuEncode	基于 JPU 硬件加速的 JPEG 图片编码插件
Gst-IxH264Encode	基于 CPU/GPU 实现的 H264 视频编码插件
Gst-IxConverter	格式转换插件
Gst-IxStreamMux	多路输入合并成一个 batch
Gst-IxStreamDemux	一个 batch 分解成对应的多路
Gst-IxPreprocess	预处理插件
Gst-IxInfer	基于 IGIE/ixRT 实现的模型推理插件

Plugin	说明
Gst-IxTracker	用于跟踪检测到的具有持久 ID 的对象
Gst-IxOsd	对输入的帧进行绘制
Gst-IxMultiStreamTiler	将多路输入图片拼接成一张图片
Gst-IxAnalytics	对Gst-IxInfer 和Gst-IxTracker 附加的元数据进行分析
Gst-IxSegVisual	对分割结果的可视化

1.6.1 IxStream SDK 中的元数据

Gst Buffer 是 GStreamer 中数据传输的基本单位，每个 Gst Buffer 都有关联的元数据。

IxStream SDK 附加的主要元数据对象如下图所示：

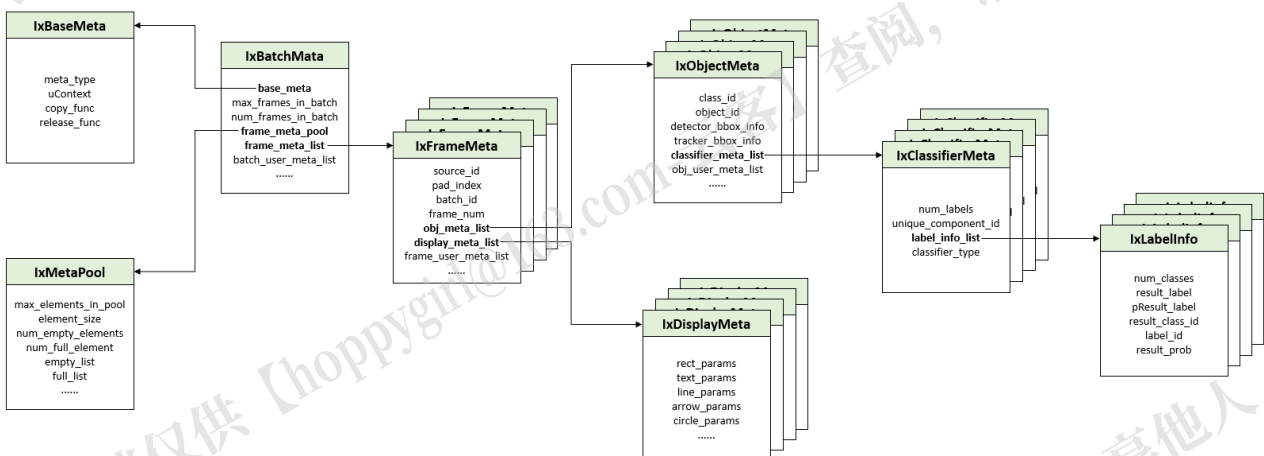


图 3: IxStream SDK 附加的主要元数据对象

您可以通过相关接口获取这些元数据 (参考[如何获取元数据](#))，同时也可以通过您自定义的接口 (如 xxx_user_meta_list) 向其中添加自定义元数据 (参考[如何添加用户自定义元数据](#))。

了解更多关于 Gst Buffer 的信息，请参考 [GStreamer 文档](#)。

1.6.1.1 如何获取元数据

C++ 获取元数据方式

1. 引入必要的头文件：

```
#include<gst/gst.h>
#include<gst_ixstream_meta.h>
```

- 通过 IxStream 对象的指针，获取该对象的 sinkpad (或 srcpad) 的指针：

```
GstPad* sink_pad = gst_element_get_static_pad(ixstream_obj, "sink");
或
GstPad* src_pad = gst_element_get_static_pad(ixstream_obj, "src");
```

- 向 sinkpad (或 srcpad) 添加 probe 函数 (指定为 GST_PAD_PROBE_TYPE_BUFFER 类型)：

```
gst_pad_add_probe(sink_pad, GST_PAD_PROBE_TYPE_BUFFER, sink_pad_buffer_probe, NULL,
↳ NULL)
```

- 在 probe 函数中获取 IxBatchMeta 对象：

```
static GstPadProbeReturn sink_pad_buffer_probe(GstPad *pad, GstPadProbeInfo *info,
↳ gpointer u_data) {
    GstBuffer *buf = (GstBuffer *) info->data;
    IxBatchMeta *pBatchMeta = gst_buffer_get_ix_batch_meta(buf);
}
```

Python 获取元数据方式

- 导入必要库：

```
import gi
gi.require_version('Gst', '1.0')
from gi.repository import GObject, Gst
import pyixstream
```

- 通过 IxStream 对象，获取该对象的 sinkpad (或 srcpad)：

```
sink_pad = ixstream_obj.get_static_pad('sink');
或
src_pad = ixstream_obj.get_static_pad('src');
```

- 向 sinkpad (或 srcpad) 添加 probe 函数 (指定为 GST_PAD_PROBE_TYPE_BUFFER 类型)：

```
sink_pad.add_probe(Gst.PadProbeType.BUFFER, sink_pad_buffer_probe, NULL)
```

- 在 probe 函数中获取 IxBatchMeta 对象：

```
def sink_pad_buffer_probe(pad, info, gpointer u_data):
    gst_buf = info.get_buffer()
    batch_meta = pyixstream.gst_buffer_get_ix_batch_meta(hash(gst_buf))
```

1.6.1.2 如何添加用户自定义元数据

C++ 添加用户自定义元数据方式

- 引入必要的头文件：

```
#include<gst/gst.h>
#include<gst_ixstream_meta.h>
```

2. 获取 IxBatchMeta 对象指针：

```
static GstPadProbeReturn sink_pad_buffer_probe(GstPad *pad, GstPadProbeInfo *info,
    ↪ gpointer u_data) {
    GstBuffer *buf = (GstBuffer *) info->data;
    IxBatchMeta *pBatchMeta = gst_buffer_get_ix_batch_meta(buf);
}
```

3. 通过 IxBatchMeta 对象，创建一个 IxUserMeta 对象：

```
IxUserMeta *pUserMeta = ix_acquire_user_meta_from_pool(pBatchMeta);
```

4. 指定 IxUserMeta 对象的拷贝函数、析构函数及数据：

```
pUserMeta->base_meta.copy_func = user_metadata_copy_func;
pUserMeta->base_meta.release_func = user_metadata_release_func;
pUserMeta->user_meta_data = user_meta_data_ptr;
```

5. 将 IxUserMeta 对象添加到相应的元数据上 (如 IxFrameMeta)：

```
ix_add_user_meta_to_frame(pFrameMeta, pUserMeta);
```

Python 添加用户自定义元数据方式

1. 导入必要库：

```
import gi
gi.require_version('Gst', '1.0')
from gi.repository import GObject, Gst
import pyixstream
```

2. 获取 IxBatchMeta 对象指针：

```
def sink_pad_buffer_probe(pad, info, gpointer u_data):
    gst_buf = info.get_buffer()
    batch_meta = pyixstream.gst_buffer_get_ix_batch_meta(hash(gst_buf))
```

3. 通过 IxBatchMeta 对象，创建一个 IxUserMeta 对象：

```
user_meta = pyixstream.ix_acquire_user_meta_from_pool(batch_meta)
```

4. 指定 IxUserMeta 对象的拷贝函数、析构函数及数据：

```
user_meta.base_meta.copy_func = user_metadata_copy_func
user_meta.base_meta.release_func = user_metadata_release_func
user_meta.user_meta_data = user_meta_data_ptr
```

5. 将 IxUserMeta 对象添加到相应的元数据上 (如 IxFrameMeta)：

```
pyixstream.ix_add_user_meta_to_frame(frame_meta, user_meta)
```

1.6.1.3 IxBatchMeta

IxStream 使用可扩展的元数据标准结构，IxBatchMeta 是其中最重要的元数据结构，可以在 **Gst-IxStreamMux Plugin** 中创建。

IxBatchMeta 包含的元素及说明如下：

- base_meta: IxStream 基础元数据结构，包含以下属性：
 - meta_type
 - uContext
 - copy_func
 - release_func
- max_frames_in_batch: 一个 batch 中可填充的最大 frame 个数，即 batch_size。该值在 **Gst-IxStreamMux Plugin** 中设置，由输入路数决定
- num_frames_in_batch: 当前 batch 中实际已填充的 frame 个数。该值在 **Gst-IxStreamMux Plugin** 中设置
 - 当出现 Timeout 时，num_frames_in_batch <= max_frame_in_batch
 - 当非 Timeout 时，num_frames_in_batch = max_frame_in_batch
- frame_meta_pool: IxFrameMetaList 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- obj_meta_pool: IxObjectMetaList 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- classifier_meta_pool: IxClassifierMetaList 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- display_meta_pool: IxDisplayMetaList 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- user_meta_pool: IxUserMetaList 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- label_info_meta_pool: label_info_meta 对应的 IxMetaPool，该 pool 在 **Gst-IxStreamMux Plugin** 中初始化
- frame_meta_list: 指向 IxFrameMetaList 的指针
- batch_user_meta_list: 指向 IxUserMetaList 的指针
- meta_mutex: MetaData 操作锁

1.6.1.4 IxFrameMeta

IxFrameMeta 包含的元素及说明如下：

- base_meta: IxStream 基础元数据结构
- pad_index: 当前 frame 对应的 sink_pad index，该值与 source_id 一一对应
- batch_id: 当前 frame 在对应 batch 中的 index
- buf_pts: 当前 frame 的 PTS，区别于 ntp_timestamp，该值由输入源 buffer 中包含的 timestamp 决定
- source_id: 当前 frame 对应的输入源 ID
- source_frame_width: 当前 frame 对应输入源的 frame_width
- source_frame_height: 当前 frame 对应输入源的 frame_height
- num_obj_meta: 当前 frame 包含的目标对应个数，该值最大为 100，超过该值赋值 100 并抛出 Warning 信息
- obj_meta_list: 指向 IxObjectMetaList 的指针

- display_meta_list: 指向 IxDisplayMetaList 的指针
- frame_user_meta_list: 指向 IxUserMetaList 的指针
- pipeline_width: streammux 后的 frame_width
- pipeline_height: streammux 后的 frame_height

1.6.1.5 IxObjectMeta

IxObjectMeta 包含的元素及说明如下:

- base_meta: IxStream 基础元数据结构
- unique_component_id: 产生当前元数据的组件唯一 ID
- object_id: 用于跟踪对象的唯一 ID
- class_id: 当前对象的 class index
- detector_bbox_info: 检测到的目标对象的 bbox 信息, 包含宽、高及位置信息
- confidence: 保存由推理组件设置的对象的置信度值
- classifier_meta_list: 指向 IxClassifierMetaList 的指针
- obj_user_meta_list: 指向 IxUserMetaList 的指针

1.6.1.6 IxClassifierMeta

IxClassifierMeta 包含的元素及说明如下:

- base_meta: IxStream 基础元数据结构
- num_labels: 分类器输出的分类总数
- unique_component_id: 产生当前元数据的组件唯一 ID
- label_info_list: 指向 IxLabelInfoList 的指针

1.6.1.7 IxLabelInfoList

IxLabelInfoList 包含的元素及说明如下:

- base_meta: IxStream 基础元数据结构
- pResult_label: 指向保存当前分类 label 信息的指针
- label_id: 当前分类的 index

1.6.1.8 IxDisplayMeta

IxDisplayMeta 包含的元素及说明如下:

- base_meta: IxStream 基础元数据结构
- num_rects: 当前帧包含的目标框的个数
- num_labels: 当前帧包含的目标 label 的个数
- num_lines: 当前帧包含的线的个数
- num_arrows: 当前帧包含的箭头个数
- num_circles: 当前帧包含的圆的个数
- rect_params: 当前帧中目标框的位置信息数组

- text_params: 当前帧中用于绘制文本框的参数信息数组
- line_params: 当前帧中用于绘制多边形的线的参数信息数组
- arrow_params: 当前帧中用于绘制箭头的参数信息数组
- circle_params: 当前帧中用于绘制圆的参数信息数组

1.6.2 Gst-IxVpuDecode

Gst-IxVpuDecode Plugin 提供视频流硬解码功能。

支持输入的视频流格式有:

- H264
- H265
- AVS2

支持输出的数据格式有:

- YUV420
- NV12

输入与输出

- 输入: H264、H265、AVS2 视频流
- 输出: 基于帧的 YUV420 数据、NV12 数据

功能

下表总结了 Gst-IxVpuDecode Plugin 的功能:

- H264 格式的硬件解码
- H265 格式的硬件解码
- AVS2 格式的硬件解码
- 对解码后的帧按一定比例丢帧
- 支持低延迟模式
- 支持 NV12 格式输出

属性

下表总结了 Gst-IxVpuDecode Plugin 的 Gst 属性:

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID, 该值需与其他 Plugin 对应值保持一致	Unsigned int, 0~4294967295, 默认值: 0
frame-width	输入视频流的宽 (需设置为输入视频对应真实值, 否则会丢弃当前帧)	Unsigned int, 0~4294967295, 默认值: 1920
frame-height	输入视频流的高 (需设置为输入视频对应真实值, 否则会丢弃当前帧)	Unsigned int, 0~4294967295, 默认值: 1080

属性	说明	类型与范围
video-format	输入视频流的格式 (需设置为输入视频对应真实值, 否则解码结果异常)	Unsigned int, {0:H264, 12:H265, 14:AVS2}, 默认值: 0
drop-frame-interval	丢帧频率 (每 N 帧输出一帧)	Unsigned int, 1~30 (大于 30 设置为 30), 默认值: 1
low-latency-mode	低延时模式, 仅对只有 IP 帧的视频有用, 禁止对有 B 帧的视频赋能	Bool, 默认值: false
output-format	输出帧数据格式 0: YUV420 1: NV12	Unsigned int, 0/1 默认值: 0

1.6.3 Gst-IxJpuDecode

Gst-IxJpuDecode Plugin 提供 JPEG 图片硬解码功能, 输入为 JPEG 类型的图片数据, 输出为 I420, RGB 或 BGR 格式数据。

输入与输出

- 输入: JPEG 图片
- 输出: 基于帧的 I420, RGB 或 BGR 数据

功能

下表总结了 Gst-IxJpuDecode Plugin 的功能:

- JPEG 格式的硬件解码

属性

下表总结了 Gst-IxJpuDecode Plugin 的 Gst 属性:

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID, 该值需与其他 Plugin 对应值保持一致, 否则抛出异常	Unsigned int, 0~4294967295, 默认值: 0
frame-width	输入 jpeg 图的宽度 (需设置为输入 jpeg 图片对应真实值)	Unsigned int, 0~4294967295, 默认值: 1920
frame-height	输入 jpeg 图的高度 (需设置为输入 jpeg 图片对应真实值)	Unsigned int, 0~4294967295, 默认值: 1080
output-format	输出数据流的数据格式 (需设置为希望输出的真实值)	Unsigned int, {1:i420, 3:rgb, 4:bgr}, 默认值为 1

1.6.4 Gst-IxJpuEncode

Gst-IxJpuEncode Plugin 提供 JPEG 图片的硬编码功能。

支持输入的视频流格式有：

- YUV400
- YUV420
- YUV422
- YUV440
- YUV444

输入与输出

- 输入：基于帧的 YUV400、YUV420、YUV422、YUV440、YUV444 格式数据
- 输出：基于帧的 JPEG 格式数据

功能

下表总结了 Gst-IxJpuEncode Plugin 的功能：

- JPEG 格式的硬件编码

属性

下表总结了 Gst-IxJpuEncode Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0
save-file-path	保存的 JPEG 文件目录 (该值必须设置，否则抛出异常)	String，无默认值

1.6.5 Gst-IxH264Encode

Gst-IxH264Encode Plugin 提供 H264 视频的编码功能 (包含 CPU 实现和 CUDA Core 加速实现)。目前支持的输入视频流格式包括 YUV420 和 NV12。

支持输入的视频流格式有：

- YUV420
- NV12

输入与输出

- 输入：基于帧的 YUV420 或 NV12 格式数据

- 输出：H264 视频文件

功能

下表总结了 Gst-IxH264Encode Plugin 的功能：

- 基于 CPU 实现的 H264 视频编码功能
- CUDA 加速的 H264 视频编码功能
- CPU 亲和性设定

属性

下表总结了 Gst-IxH264Encode Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0
save-video -path	保存的视频文件目录绝对路径	String，无默认值
save-video -name	保存的视频文件的文件名	String，默认值：h264out.h264
threads-num	CPU 编码部分并行线程数	Unsigned int，1~128，默认值：1
use-cuda	是否使用 CUDA 加速	Bool，true/false，默认值：false
cpu-affinity	CPU 亲和性 (以","号进行分割)	String，默认值：all
enable-sink	是否在当前插件保存编码后的视频 (为了兼容以前的 demo 样例，目前设置默认值为 true，即在当前插件保存文件，后续会弃用属性，需在 pipeline 中接入 filesink 保存数据)	Bool，true/false，默认值为 true

1.6.6 Gst-IxConverter

Gst-IxConverter Plugin 提供基于帧数据的格式转换功能。

支持以下格式间的转换：

- I420
- NV12
- NV21
- RGB
- BGR

输入与输出

- 输入：基于帧的 I420、NV12、NV21、RGB 格式数据
- 输出：基于帧的 I420、NV12、NV21、RGB、BGR 格式数据

功能

下表总结了 Gst-IxConverter Plugin 的功能：

- I420 和 RGB 格式间转换
- 支持 NV12 格式
- 支持 NV21 格式
- 支持 BGR 输出

属性

下表总结了 Gst-IxConverter Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int, 0 ~4294967295，默认值：0
drop-only	用于设置当发生掉帧的情况如何处理时间戳 - True: 只保持原时间戳发送，不补充帧 - False: 复制前一帧补充 drop 的帧	Boolean，默认值：True

1.6.7 Gst-IxStreamMux

Gst-IxStreamMux Plugin 将多个输入源的数据合并成一个 batch 数据。如果输入源的分辨率不同，会对输入做 resize 并统一成同一分辨率 (可以使用 width 和 height 属性指定此分辨率)。如果 batch 中数据已满或超时达到 batched-push-timeout (收集到 batch 中的第一个缓冲区时，超时开始运行) 时，Gst-IxStreamMux 会将 batch 数据向下游推送。当 live-source 属性设置为 true 时，Gst-IxStreamMux 将执行实时非阻塞策略 (输出 buffer 满则自动丢弃，该策略适用于 rtsp 等实时输入流)。当 live-source 属性设置为 false 时，Gst-IxStreamMux 将执行阻塞策略 (输出 buffer 满则进行等待，该策略适用于视频文件输入流)。

输入与输出

- 输入：Gst buffer
- 输出：Gst buffer

功能

下表总结了 Gst-IxStreamMux Plugin 的功能：

- 可配置的批处理超时
- 允许具有不同分辨率的多个输入流
- 允许具有不同帧率的多个输入流
- 可以 resize 到用户指定的分辨率

属性

下表总结了 Gst-IxStreamMux Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0
width	mux 后的帧的 width	Unsigned int，0~4294967295，默认值：1920
height	mux 后的帧的 height	Unsigned int，0~4294967295，默认值：1080
live-source	输入源是否为实时源	Bool，true/false，默认值：true
batched-push-timeout	超时时间，单位为 ms	Unsigned int，0~4294967295，默认值：4294967295
enable-padding	resize 操作是否执行 padding	Bool，true/false，默认值：false

1.6.8 Gst-IxStreamDemux

Gst-IxStreamDemux Plugin 将 batch 帧 demux 到单独的缓冲区中。它为 batch 中的每个帧创建一个单独的 Gst 缓冲区，它不复制视频帧。每个 Gst 缓冲区都包含一个指向 batch 中相应帧的指针。

输入与输出

- 输入：Gst buffer
- 输出：Gst buffer

功能

下表总结了 Gst-IxStreamDemux Plugin 的功能：

- 根据输入路数将 batched 输入分成多路输出

属性

下表总结了 Gst-IxStreamDemux Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0

1.6.9 Gst-IxPreprocess

Gst-IxPreprocess Plugin 对输入的帧数据做 infer 前的预处理操作。该插件可以在 PGIE 和 SGIE 两种模式下运行。

PGIE 模式预处理用于处理给定的帧，需要在其上执行初步推理；SGIE 模式预处理用于处理给定帧内检测到的对象，需要对其执行二次推理。

输入与输出

- 输入：Gst buffer、元数据
- 输出：Gst buffer、元数据

功能

下表总结了 Gst-IxPreprocess Plugin 的功能：

- 支持 PGIE 模式
- 支持 SGIE 模式

属性

下表总结了 Gst-IxPreprocess Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0
enable	用于使能Gst-IxPreprocess 插件，当设置为 false 时，Gst-IxPreprocess 插件不对输入的 buffer 做任何处理，直接向下传递	bool，true/false，默认值为 true
config-file	配置文件绝对路径，该值必须设置，否则抛出异常	String，无默认值
operate-on-gie-id	此处与此 ID 匹配的 GIE 生成的元数据，当为-1 时，会处理所有 GIE 产生的数据	Int，-1~4294967295，默认值为-1
process-on-frame	用以指定Gst-IxPreprocess 处理对象，Frame 或 Object (true: Frame, false: Object)	Bool，默认值为 true
target-unique-ids	指定处理Gst-IxPreprocess 输出的 GIE 的 ID，GIE 的 ID 以分号分隔组成字符串，如"3;4;5"	String，无默认值

属性	说明	类型与范围
unique-id	元素的唯一 ID，用于区分输出元素	Unsigned Int, 0~4294967295, 默认值为 15

配置文件说明

表 13: Gst-IxPreprocess 配置文件说明

参数	说明	类型与范围
target-unique-ids	指定处理Gst-IxPreprocess 输出的 GIE 的 ID，GIE 的 ID 以分号分隔组成字符串，如"3;4;5"	String，无默认值
operate-on-gie-id	处理与此 ID 匹配的 GIE 生成的数据，当为-1 时，会处理所有 GIE 产生的数据	Int, -1~4294967295, 默认值为-1
enable	用于使能Gst-IxPreprocess 插件，当设置为 false 时，Gst-IxPreprocess 插件不对输入的 buffer 做任何处理，直接向下传递	bool, true/false, 默认值为 true
unique-id	元素的唯一 ID，用于区分输出元素	Unsigned Int, 0~4294967295, 默认值为 15
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned Int, 0~4294967295, 默认值为 0
process-on-frame	用于指定Gst-IxPreprocess 处理对象，Frame 或 Object (true: Frame, false: Object)	Bool, 默认值为 true
processing-width	ROI 放缩后的宽，该属性需要主动设置，否则抛出异常	Unsigned Int, 0~4294967295, 默认值为 640
processing-height	ROI 放缩后的高，该属性需要主动设置，否则抛出异常	Unsigned Int, 0~4294967295, 默认值为 480
maintain-aspect-ratio	是否保持 ROI 放缩后的纵横比	Bool, true/false, 默认值为 true
symmetric-padding	设置后将进行对称填充	Bool, true/false, 默认值为 true
tensor-buf-pool-size	tensor buffer pool (用于存储Gst-IxPreprocess 处理结果) 的大小	Int, 0~4294967295, 默认值为 6

参数	说明	类型与范围
scaling-buf-pool-size	scaling buffer pool (用于存储 ROI 放缩后的结果) 的大小	Int, 0~4294967295, 默认值为 6
scaling-filter	指定放缩时的插值算法 (目前只支持 IxBufSurfTransformInter_Bilinear), 有 0=IxBufSurfTransformInter_Nearest, 1=IxBufSurfTransformInter_Bilinear, 2=IxBufSurfTransformInter_Algo, 3=IxBufSurfTransformInter_Algo2, 4=IxBufSurfTransformInter_Algo3, 5=IxBufSurfTransformInter_Algo4, 6=IxBufSurfTransformInter_Default, 该属性需要主动设置, 否则抛出异常	Int, [0,6], 默认值为 6
scaling-pool-memory-type	scaling buffer pool 的内存类型, 有 0=IXBUF_MEM_DEFAULT, 1=IXBUF_MEM_CUDA_PINNED, 2=IXBUF_MEM_CUDA_DEVICE, 3=IXBUF_MEM_CUDA_UNIFIED, 4=IXBUF_MEM_SYSTEM, 该属性需要主动设置, 否则抛出异常	Int, [0,4], 默认值: 0
network-input-order	网络输入层的 order, 0=NCHW, 1=NHWC, 该属性需要主动设置, 否则抛出异常	Int, [0,1], 默认值为 0
network-input-shape	网络输入层的 shape, 该属性需要主动设置, 否则抛出异常	以分号分割的整型数组, 如 [60, 3, 368, 640], 无默认值
network-color-format	模型的色彩格式, 0=RGB, 1=BGR, 2=GRAY, 该属性需要主动设置, 否则抛出异常	Int, [0,2], 默认值为 0
tensor-data-type	推理时的数据格式, 0=FP32, 1=UINT8, 2=INT8, 3=UINT32, 4=INT32, 5=FP16, 该属性需要主动设置, 否则抛出异常	Int, 默认值 0
tensor-name	tensor 名称, 该值必须设置, 否则抛出异常	String, 无默认值
custom-lib-path	自定义库.so 文件的路径, 该属性需要主动设置, 否则抛出异常	String, 无默认值
custom-tensor-preparation-function	自定义库中 tensor 预处理函数名称 (用于 ROI 区域的缩放), 该属性需要主动设置, 否则抛出异常	String, 无默认值
pixel-normalization-factor	指定像素值缩放因子	Float, 无默认值

参数	说明	类型与范围
offsets	存放均值的数组，数组的长度必须与 frame 中色彩通道数匹配	Array (Int)，无默认值
mean-file	均值数据文件路径名 (PPM 格式)	String，无默认值
src-ids	该 group 相关的源 IDs，该属性需要主动设置，否则抛出异常	Array(Int)，以逗号分割，如 [1, 2, 3]，无默认值
draw_roi	ROI 对象是否画框，该属性需要主动设置，否则抛出异常	Bool，true/false，默认值：true
roi_color	ROI 对象画框颜色，该属性需要主动设置，否则抛出异常	Array(Float)
process-on-roi	设置为 true 时，处理的对象为 ROI，否则为整个 frames，该属性需要主动设置，否则抛出异常	Bool，true/false，默认值为 false
process-on-all-objects	设置为 true 时，会处理所有 Objects，否则只处理 ROI 区域内的 Objects，该属性需要主动设置，否则抛出异常	Bool，true/false，默认值为 false
roi-params-src	ROI 区域的坐标，数组格式，每个 ROI 区域由 left、top、width、height 指定，可指定多个 ROI 区域	Array(Int)，如 [0,540,900,500,960,0,900,500]
custom-input-transformation-function	自定义库中自定义输入转换函数的名称 (用于 ROI 区域的缩放)	String，无默认值
operate-on-class-ids	指定需要处理的类型 ID (当 process-on-frame 为 true 时，该属性无效)，该属性需要主动设置，否则抛出异常	Array(Int)，如 [1, 3, 4]
input-object-min-width	将要处理的 Objects 的最小宽度，该属性需要主动设置，否则抛出异常	Int，0~4294967295，无默认值
input-object-min-height	将要处理的 Objects 的最小高度，该属性需要主动设置，否则抛出异常	Int，0~4294967295，无默认值
input-object-max-width	将要处理的 Objects 的最大宽度，该属性需要主动设置，否则抛出异常	Int，0~4294967295，无默认值
input-object-max-height	将要处理的 Objects 的最大高度，该属性需要主动设置，否则抛出异常	Int，0~4294967295，无默认值

1.6.10 Gst-IxInfer

Gst-IxInfer Plugin 使用 IGIE 推理框架/ixRT 推理引擎对输入数据进行推理。

Gst-IxInfer Plugin 目前支持以下网络的推理：

- 多类物体检测，已支持以下模型：
 - YOLOv3
 - YOLOv4

- YOLOv5
- YOLOv6
- YOLOv7
- YOLOv8
- YOLOv9
- YOLOv10
- YOLOX
- 多标签分类
- 图像语义分割，已支持 unet 模型

不同模型需要通过 parse-bbox-func-name 属性来指定后处理函数。

IxStream 提供了 libgst_ixstream_infer_custom_impl_Yolo.so 库以对常见 YOLO 模型的后处理进行封装，当前已提供的 YOLO 后处理函数及说明如下表所示。

对于其它未提供后处理函数的模型，您可以通过 IxStream 提供的自定义接口编写自己的后处理实现 (参照 ixstream_samples/C++/ixstream_sample_postprocess)。

表 14: YOLO 后处理函数及说明

模型	后处理函数名	模型出处
YOLOv3	IxInferParseCustomYoloV3	Gitee YOLOv3
YOLOv4	IxInferParseCustomYoloV4	Gitee YOLOv4
YOLOv5	IxInferParseCustomYoloV5	Gitee YOLOv5
YOLOv6	IxInferParseCustomYoloV6	Gitee YOLOv6
YOLOv7	IxInferParseCustomYoloV7	Gitee YOLOv7
YOLOv8	IxInferParseCustomYoloV8	Gitee YOLOv8
YOLOv9	IxInferParseCustomYoloV9	Gitee YOLOv9
YOLOv10	IxInferParseCustomYoloV10	Gitee YOLOv10
YOLOX	IxInferParseCustomYoloX	Gitee YOLOX
YOLO with NMS	IxInferParseCustom4EngineWithNMS	带 NMS 加速的 YOLO 模型实现

Gst-IxInfer Plugin 可以在两种模式下工作：

- primary：在全帧上运行 (比如在一级推理中对所有输入的帧进行目标对象检测)
- secondary：对上游组件添加到元数据中的对象进行操作 (比如在二级推理中对上游一级推理中检测到的目标对象进行进一步分类)

输入与输出

- 输入：Gst buffer、元数据
- 输出：Gst buffer、元数据

功能

下表总结了 Gst-IxInfer Plugin 的功能：

- 支持 primary 模式
- 支持 secondary 模式
- 支持 IGIE Infer
- 支持 ixRT Infer

属性

下表总结了 Gst-IxInfer Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值：0
config-file-path	配置文件绝对路径，该值必须设置，否则抛出异常	String，无默认值

配置文件说明

表 16: Gst-IxInfer 配置文件说明

参数	说明	类型与范围
process-mode	推理模式，1: primary; 2: secondary	Int, 1/2, 默认值: 1
network-type	网络的类型, 0: Detector, 1: Classifier, 2: Segmentation	Int, 默认值: 0
model-engine-file	IGIE/ixRT 生成的模型引擎文件，该值必须设置，否则抛出异常	String, 无默认值
input-blob-name	模型输入名，必须设置正确的输入名	String, 无默认值
output-blob-names	模型输出名，必须设置正确的输出名	String, 无默认值
infer-dims	模型输入维度 (C,H,W) 信息，该值必须设置，否则抛出异常	Vector3_i, 默认值: {0, 0, 0}
network-input-order	模型输入类型，0: NCHW; 1: NHWC	Int, 0/1, 默认值: 0
batch-size	模型 batch 大小，该值必须设置，否则抛出异常	Unsigned Int, 0~4294967295, 默认值: 0
net-scale-factor	模型输入图片缩放因子，用于预处理	Float, >0.0, 默认值: 1.0

参数	说明	类型与范围
offsets	模型输入图片偏移因子，用于预处理	Vector3_f, 默认值： {0.0,0.0,0.0}
maintain-aspect-ratio	resize 操作是否保持纵横比，用于预处理	Bool, true/false, 默认值： TRUE
symmetric-padding	resize 操作是否执行对称填充，用于预处理	Bool, true/false, 默认值： FALSE
num-detected-classes	检测网络的类别数，该值必须设置，否则抛出异常	Unsigned Int, 0~4294967295, 默认值： 0
parse-bbox-func-name	自定义边界框解析函数名称，用于后处理，该值必须设置，否则抛出异常	String, 无默认值
parse-classifier-func-name	自定义分类输出解析函数名称，用于后处理，该值必须设置，否则抛出异常	String, 无默认值
custom-lib-path	自定义的模型后处理库的路径，该值必须设置，否则抛出异常	String, 无默认值
filter-out-class-ids	过滤掉检测到的属于指定 class-id 的对象，如果为空则不进行过滤	Vector<gint>, 无默认值
operate-on-class-ids	对上游模型检测到的指定 class-ids 的对象进行操作	Unsigned Int, 0~4294967295, 默认值： 0
labelfile-path	class 与 label 的映射文件，支持的分隔符有：分号，回车	String, 无默认值
input-tensor-from-meta	使用经过预处理的输入 tensor (通过 metadata 附加在输入上)，取代Gst-IxInfer 内部的预处理	Bool, true/false, 默认值： FALSE
input-object-min-width	二级推理中能够处理的对象的最小宽度	Unsigned Int, >=0, 默认值： 0
input-object-min-height	二级推理中能够处理的对象的最小高度	Unsigned Int, >=0, 默认值： 0
input-object-max-width	二级推理中能够处理的对象的最大宽度	Unsigned Int, >=0, 默认值： 4294967295
input-object-max-height	二级推理中能够处理的对象的最大高度	Unsigned Int, >=0, 默认值： 4294967295
model-color-format	模型所需要的颜色类型 (若 input-tensor-from-meta 为 True, 则可忽略)	Int, 0: RGB, 1: BGR; 默认值: 0
classifier-threshold	标签概率的最小阈值	Float, >=0.0, 默认值: 0.0

参数	说明	类型与范围
output-tensor-meta	使能后， Gst-IxInfer 会将未经处理的推理结果以 metadata 的形式附加到输出 buffer 上	Bool，默认值：False
cluster-mode	用于指定聚类算法的类型	Int，默认值：2，2：Non Maximum Suppression，4：No clustering

1.6.11 Gst-IxTracker

Gst-IxTracker Plugin 用于跟踪检测到的具有持久 ID 的对象。

当前已实现的 tracker 算法如下：

- deepsort

输入与输出

- 输入：Gst buffer、元数据
- 输出：Gst buffer、元数据

功能

下表总结了 Gst-IxTracker Plugin 的功能：

- 支持 deepsort 算法

属性

下表总结了 Gst-IxTracker Plugin 的 Gst 属性：

表 17: Gst-IxTracker Plugin 的 Gst 属性

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int，0~4294967295，默认值为 0
config-file-path	配置文件绝对路径，该值必须设置，否则抛出异常	String，无默认值
ll-lib-file	底层跟踪库的路径名，IxStream 提供一个默认的跟踪库 /usr/local/corex/lib64/libixstream_multiobjecttracker.so	String，无默认值

配置参数说明

下表总结了 Gst-IxTracker 中常见模块的配置参数：

模块	参数	说明	类型与范围	默认值	备注
Target Management	probationAge	试用期长度 (以帧数为单位)	Int, ≥ 0	5	
Target Management	maxShadowTrackingAge	阴影跟踪最大长度	Int, ≥ 0	38	
ReID	reidType	Re-ID 网络的类型 {DUMMY=0, NvDEEPSORT=1, Reid based reassoc=2, DEEPSORT and reid based reassoc=3}	Int, [0, 3]	0	
ReID	batchSize	Re-ID 网络的 batch 大小	Int, > 0	1	
ReID	reidFeatureSize	Re-ID 的特征大小	Int, > 0	128	
ReID	reidHistorySize	特征库的大小, 即为一个跟踪器保留的 Re-ID 特征的最大数量	Int, > 0	100	
ReID	resize	Re-ID 网络预处理时 resize 的尺寸信息 (包含 W/H 两个维度)	Vector2_i	[0, 0]	当设置该属性时, 先 resize 成对应大小再 crop 成 inferDims 的输入, 否则直接 resize 成 inferDims 的输入
ReID	inferDims	Re-ID 网络基于 inputOrder 的输入维度信息 (仅包含 C/H/W 三个维度)	Vector3_i	[128, 64, 3]	
ReID	networkMode	Re-ID 网络推理精度 {FP32=0, FP16=1, INT8=2}	Int, [0, 2]	0	

模块	参数	说明	类型与范围	默认值	备注
ReID	inputOrder	Re-ID 网络模型输入类型 {NCHW=0, NHWC=1}	Int, [0, 1]	0	
ReID	colorFormat	Re-ID 网络输入颜色格式 {RGB=0, BGR=1}	Int, [0, 1]	0	
ReID	offsets	Re-ID 网络输入数据的偏移值 (基于每个颜色通道)	Vector3_f	[0.0, 0.0, 0.0]	
ReID	netScaleFactor	Re-ID 网络输入的缩放因子	Float, >0.0	1.0	
ReID	normalize	Re-ID 网络输入的标准化参数	Vector3_f	[1.0, 1.0, 1.0]	
ReID	keepAspc	调整输入对象大小到 Re-ID 网络时是否保持长宽比	Bool, true/false	true	
ReID	inputBlobName	Re-ID 网络输入层名称	String	"images"	
ReID	outputBlobName	Re-ID 网络输出层名称	String	"features"	
ReID	modelEngineFile	IGIE/ixRT 生成的 Re-ID 模型引擎文件绝对路径	String	""	

1.6.12 Gst-IxOsd

Gst-IxOsd Plugin 可以对输入的帧绘制边界框、文本、箭头、线条、圆形和多边形。

该 Plugin 支持以下两种处理模式：

- CPU 处理模式 (通过 OpenCV 库实现, 不支持中文字体)
- GPU 处理模式 (通过 cuOSD 库实现, 支持中文字体)

输入与输出

- 输入: Gst buffer、元数据
- 输出: Gst buffer、元数据

功能

下表总结了 Gst-IxOsd Plugin 的功能：

- 支持 CPU 处理模式
- 支持 GPU 处理模式
- 支持绘制箭头
- 支持绘制圆
- 支持绘制边界框
- 支持绘制多边形
- 支持绘制文本
- GPU 处理模式，支持中文字体

属性

下表总结了 Gst-IxOsd Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与其他 Plugin 对应值保持一致，否则抛出异常	Unsigned int, 0~4294967295, 默认值：0
process-mode	控制相关操作使用的硬件平台	Unsigned int, {0:CPU, 1:GPU}, 默认值：0
display-bbox	是否显示边界框	Bool, true/false, 默认值：false
display-text	是否显示文本	Bool, true/false, 默认值：false
font-file-path	中文字体路径，仅在 GPU 模式下支持	String, 默认值：/usr/share/fonts/ixstream/WenQuanYiMicroHei.ttf

1.6.13 Gst-IxMultiStreamTiler

Gst-IxMultiStreamTiler Plugin 可以将多个输入源的图片数据拼接成一个图片数据。拼接后的图片大小由属性 width 和 height 指定。拼接后的图片布局遵循以下原则：

- 行优先原则
- 当输入源个数 $\leq$ rows*columns 时，不足的位置做 padding
- 当输入源个数 $>$ rows*columns 时，忽略传入的 rows 和 columns 参数，重新对 rows 和 columns 数进行设置，设置规则如下：
 - rows = (int)(sqrt(input_channels))
 - columns = (int)((input_channels + rows - 1) / rows)

输入与输出

- 输入：Gst buffer
- 输出：Gst buffer

功能

下表总结了 Gst-IxMultiStreamTiler Plugin 的功能：

- 将多个输入源的图片数据拼接成一个图片数据

属性

下表总结了 Gst-IxMultiStreamTiler Plugin 的 Gst 属性：

属性	说明	类型与范围
gpu-id	用于操作的 GPU ID，该值需与跟其他 Plugin 对应值保持一致，否则抛出异常	Unsigned Int，0~4294967295，默认值：0
width	拼接后的帧的 width	Unsigned int，0~4294967295，默认值：1920
height	拼接后的帧的 height	Unsigned int，0~4294967295，默认值：1080
rows	拼接的行数	Unsigned int，0~4294967295，默认值：1
columns	拼接的列数	Unsigned int，0~4294967295，默认值：1

1.6.14 Gst-IxAnalytics

Gst-IxAnalytics Plugin 用于对 **Gst-IxInfer** 和 **Gst-IxTracker** 附加的元数据进行分析，分析的对象包括：目标区域 (ROI) 过滤、过度拥挤检测、方向检测和越线检测。

基于 ROI 的分析适用于 **Gst-IxInfer** 输出，但方向检测和越线需要 Tracker ID，因为执行这两项分析都需要以前的历史记录/状态。分析规则的配置是使用配置文件实现的。对所有分析计算，都是用对象边界框的底部中心坐标。如果边界框定义为 (x_left, y_top, width, height)，则底部中心坐标将为 (x_left + width/2, y_top + height)。

输入与输出

- 输入：Gst buffer、元数据
- 输出：Gst buffer、元数据

功能

下表总结了 Gst-IxAnalytics Plugin 的功能：

- 目标区域 (ROI) 过滤：此功能检测 ROI 中是否存在对象，以及每帧 ROI 中的对象总数。检测结果作为每个对象的元数据附加
- 过度拥挤检测：此功能检测 ROI 中是否存在对象，以及每帧 ROI 中的对象总数以及过度拥挤状态，即 ROI 中的对象没有超过预先配置的阈值。检测结果作为每个对象的元数据附加

- 方向检测：使用对象位置历史记录和当前帧信息检查对象是否遵循预先配置的方向。检测结果作为每个对象的元数据附加
- 越线检测：此功能检查对象是否遵循虚拟线的预配置方向以及是否跨越虚拟线。检测结果作为每个对象的元数据附加

属性

下表总结了 Gst-IxAnalytics Plugin 的 Gst 属性：

属性	说明	类型与范围	备注
enable	是否开启分析功能	Bool，默认值为 1	
unique-id	元素的唯一标识无符号整数值，用于标识元素附加的输出	Int，默认值为 15	预留接口，暂未实现
config-file-path	配置文件绝对路径，该值必须设置，否则抛出异常	String，无默认值	

配置参数说明

Gst-IxAnalytics 配置参数包含以下模块属性组：

- property：基础属性组，该属性组必须配置，否则抛出异常
- roi-filtering：目标区域 (ROI) 过滤相关属性组，该属性组为动态配置项，支持多组配置，通过 stream-N 标签进行分组设置
- overcrowding：过度拥挤检测相关属性组，该属性组为动态配置项，支持多组配置，通过 stream-N 标签进行分组设置
- direction-detection：方向检测相关属性组，该属性组为动态配置项，支持多组配置，通过 stream-N 标签进行分组设置
- line-crossing：越线检测相关属性组，该属性组为动态配置项，支持多组配置，通过 stream-N 标签进行分组设置

下表总结了 Gst-IxAnalytics 中常见模块的配置参数：

表 22: Gst-IxAnalytics 中常见模块的配置参数

参数	模块	说明	类型与范围	默认值
enable	property, roi-filtering, overcrowding, direction-detection, line-crossing	是否启用相关功能	Bool, true/false	false
config-width	property	配置分析规则的宽度	Int, >0	1920
config-height	property	配置分析规则的高度	Int, >0	1080

参数	模块	说明	类型与范围	默认值
osd-mode	property	0: 禁用任何行、ROIS 和文本的显示 1: 仅显示行、ROIS 和静态文本, 即标签 2: 显示所有信息	Int, [0,1,2]	2
display-font-size	property	显示的字体大小	Int, >0	12
display-obj-cnt	property	是否显示目标对象的统计信息	Bool, true/false	false
roi-RF, roi-OC	roi-filtering, overcrowding	用于配置 RF 和 OC 的多边形目标区域, 用户可通过至少三个点的坐标进行配置, 配置的点需按多边形顺序指定	List	无
inverse-roi	roi-filtering	如果该标志设置为 true, 则表示配置的目标区域为 ROI 之外的区域	Bool, true/false	false
class-id	roi-filtering, overcrowding, direction-detection, line-crossing	分析的 ID 对象, 设置为-1 表示所有对象	List	无
object-threshold	overcrowding	过度拥挤指标的阈值	Int, >=0	无
direction-<label>	direction-detection	用于配置方向, 当目标对象按照配置的方向移动时, label 字符串将会作为用户元数据附加到对象上	List	无
line-crossing-<label>	line-crossing	该值配置为两个用于表示方向的坐标, 后接两个用于表示虚拟线的表标。当目标对象按照配置的方向移动且越线后, label 字符串将会作为用户元数据附加到对象上	List	无
extended	line-crossing	越线检测是否包含虚拟线的延长线	Bool, true/false	false
mode	line-crossing	用于表示越线检测方向一致性的规则, 可选的规则包括: strict, balanced, loose	String	loose

1.6.15 Gst-IxSegVisual

Gst-IxSegVisual Plugin 用于对分割结果的可视化, 分割是基于图像识别的, 只不过分类发生在像素级, 而不是像图像识别那样发生在图像级。分割输出大小通常与输入大小相同。

输入与输出

- 输入：Gst buffer、元数据
- 输出：Gst buffer

功能

下表总结了 Gst-IxSegVisual Plugin 的功能：

- 依据类别为分割后的数据分配不同的颜色：此插件仅显示分割输出，不会将输出叠到原始输入图上

属性

下表总结了 Gst-IxSegVisual Plugin 的 Gst 属性：

属性	说明	类型与范围
batch-size	推理的最大批处理大小	Unsigned Int, 0~4294967295, 默认值为 1
gpu-id	用于操作的 GPU ID, 该值需与其他 Plugin 对应值保持一致, 否则抛出异常	Unsigned Int, 0~4294967295, 默认值为 0
height	输出帧的 height	Unsigned Int, 0~4294967295, 默认值为 720
operate-on-seg-meta-id	处理与此 ID 匹配的元数据, 当为-1 时, 会处理所有数据	Int, -1~4294967295, 默认值为-1
unique-id	元素的唯一 ID, 用于区分输出元素	Unsigned Int, 0~4294967295, 默认值为 0
width	输出帧的 width	Unsigned Int, 0~4294967295, 默认值为 1280

1.7 IxStream Python API 参考

1.7.1 数据结构

1.7.1.1 pyixstream.IxBufSurfaceColorFormat

描述

(枚举) 图像格式的枚举值

数据字段

IXBUF_COLOR_FORMAT_INVALID	0 图像格式无效
IXBUF_COLOR_FORMAT_YUV420	1 图像格式为 YUV420
IXBUF_COLOR_FORMAT_NV12	2 图像格式为 NV12
IXBUF_COLOR_FORMAT_NV21	3 图像格式为 NV21
IXBUF_COLOR_FORMAT_RGB	4 图像格式为 RGB(NCHW)
IXBUF_COLOR_FORMAT_BGR	5 图像格式为 BGR(NCHW)
IXBUF_COLOR_FORMAT_RGBx	6 图像格式为 RGBx(NCHW)
IXBUF_COLOR_FORMAT_BGRx	7 图像格式为 BGRx(NCHW)
IXBUF_COLOR_FORMAT_R32F_G32F_B32F	8 图像格式为 RGB 类型的 float 数据 (NCHW)
IXBUF_COLOR_FORMAT_GRAY8	9 图像格式为 GRAY
IXBUF_COLOR_FORMAT_GRAY32F	10 图像格式为 GRAY 类型的 float 数据
IXBUF_COLOR_FORMAT_LAST	11 无具体指向，最后一个枚举值

1.7.1.2 pyixstream.IxBufSurfaceMemType

描述

(枚举) 内存类型

数据字段

IXBUF_MEM_DEFAULT	0 内存类型为默认值 (IXBUF_MEM_CUDA_DEVICE)
IXBUF_MEM_CUDA_PINNED	1 内存类型为 GPU 锁页内存

IXBUF_MEM_CUDA_DEVICE	2 内存类型为 GPU 内存
IXBUF_MEM_CUDA_UNIFIED	3 内存类型为 GPU 统一内存
IXBUF_MEM_SYSTEM	4 内存类型为 CPU 内存

1.7.1.3 pyixstream.IxBufSurfaceParams

描述

(结构体) 保存 batch 内存中单个图像的内存信息

数据字段

width	guint, 图像的宽度
height	guint, 图像的高度
IxBufSurfaceColorFormat colorFormat	IxBufSurfaceColorFormat, 图像的格式
dataSize	guint, 图像的大小
dataPtr	图像存储的内存指针

转换函数

将给定的对象或数据转换为 IxBufSurfaceParams 格式, 调用方式为 pyixstream.IxBufSurfaceParams.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.4 pyixstream.IxBufSurface

描述

(结构体) 保存 batch 内存的信息

数据字段

gpuId	guint 类型, batch 内存所在的 GPU 编号
batchSize	guint 类型, batch 的大小
numFilled	guint 类型, batch 内存中实际填充的内存数量
isContiguous	gboolean, batch 内存中所有的内存是否连续的标志位

memType	IxBufSurfaceMemType 枚举, batch 内存的内存类型
surfaceList	IxBufSurfaceParams 列表, batch 内存中存储的图像列表

转换函数

将给定的对象或数据转换为 IxBufSurface 格式, 调用方式为 `pyxistream.IxBufSurface.cast(data)`

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.5 pyxistream.IxMetaType

描述

(枚举) 元数据类型

数据字段

IX_INVALID_META	无效的元数据
IX_BATCH_META	batch 元数据
IX_FRAME_META	帧元数据
IX_OBJ_META	目标元数据
IX_DISPLAY_META	显示元数据
IX_CLASSIFIER_META	分类器元数据
IX_LABEL_INFO_META	标签元数据
IX_USER_META	用户元数据
IX_PAYLOAD_META	负荷元数据
IX_EVENT_MSG_META	事件消息元数据
IX_OPTICAL_FLOW_META	光流元数据
IX_LATENCY_MEASUREMENT_META	延迟测量元数据
IXINFER_TENSOR_OUTPUT_META	推理张量输出元数据
IXINFER_SEGMENTATION_META	分割网络元数据
IX_CROP_IMAGE_META	编码目标的裁剪元数据
IX_TRACKER_PAST_FRAME_META	跟踪过去帧设置的元数据
IX_AUDIO_BATCH_META	音频 batch 元数据
IX_AUDIO_FRAME_META	音频帧元数据

IX_PREPROCESS_FRAME_META	指定预处理帧元数据
IX_PREPROCESS_BATCH_META	指定预处理 batch 元数据
IX_CUSTOM_MSG_BLOB	经典信息块的元数据
IX_RESERVED_META	保留元数据
IX_GST_CUSTOM_META	指定枚举类型的经典值
IX_START_USER_META	用户元数据的起始值
IX_PREPROCESS_BATCH_OBJ_META	指定预处理 batch 的目标元数据

1.7.1.6 pyixstream.IxOSD_Mode

描述

(枚举) 绘制图像使用的设备平台

数据字段

MODE_CPU	0 调用 opencv 库绘制
MODE_GPU	1 调用 cuosd 库绘制
MODE_NONE	2 不支持的平台

1.7.1.7 pyixstream.IxOSD_Arrow_Head_Direction

描述

(枚举) 在图像中绘制箭头时的方向值

数据字段

START_HEAD	0 箭头在坐标的起始处
END_HEAD	1 箭头坐标的终止处
BOTH_HEAD	2 双向箭头

1.7.1.8 pyixstream.IxOSD_ColorParams

描述

(结构体) 在图像中绘制时绘制对象的颜色参数 (颜色参数为 RGBA，值的范围为 0.0 至 1.0)

数据字段

double red	色域空间中的 R 分量
double green	色域空间中的 G 分量
double blue	色域空间中的 B 分量
double alpha	色域空间中的 A 分量

转换函数

将给定的对象或数据转换为 IxOSD_ColorParams 格式,调用方式为 pyixstream.IxOSD_ColorParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

设置函数

为需要绘制的对象设置颜色，调用方式为 pyixstream.IxOSD_ColorParams.cast(data)

```
set(self: pyixstream.IxOSD_ColorParams, red: float, green: float, blue: float, alpha:
↪ float)-> None
```

1.7.1.9 pyixstream.IxOSD_FontParams

描述

(结构体) 在图像中绘制文本时文本的字体参数

数据字段

char *font_name	str, 字体名称
unsigned int font_size	unsigned int, 字体大小
IxOSD_ColorParams font_color	IxOSD_ColorParams, 字体颜色

转换函数

将给定的对象或数据转换为 IxOSD_FontParams 格式,调用方式为 pyixstream.IxOSD_FontParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.10 pyixstream.IxOSD_TextParams

描述

(结构体) 在图像中绘制文本时文本的参数

数据字段

display_text	str, 绘制的文本
x_offset	unsigned int, 文本所在位置的 x 坐标
y_offset	unsigned int, 文本所在位置的 y 坐标
font_params	IxOSD_FontParams, 文本的字体参数
int set_bg_clr	int, 设置背景颜色标志
text_bg_clr	IxOSD_ColorParams, 背景颜色

转换函数

将给定的对象或数据转换为 IxOSD_TextParams 格式,调用方式为 pyixstream.IxOSD_TextParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.11 pyixstream.IxOSD_RectParams

描述

(结构体) 在图像中绘制矩形框时的矩形框参数

数据字段

left	float, 矩形框左上顶点的 x 坐标
top	float, 矩形框左上顶点的 y 坐标
width	float, 矩形框的宽度
height	float, 矩形框的高度
border_width	unsigned int, 矩形框边框线的宽度
border_color	IxOSD_ColorParams, 矩形框边框线的颜色
has_bg_color	unsigned int, 矩形框填充标志

reserved	unsigned int, 保留字段
bg_color	IxOSD_ColorParams, 矩形框的填充颜色

转换函数

将给定的对象或数据转换为 IxOSD_RectParams 格式,调用方式为 pyixstream.IxOSD_RectParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.12 pyixstream.IxOSD_LineParams

描述

(结构体) 在图像中绘制线条时的参数

数据字段

x1	unsigned int, 线条起始坐标中的 x 坐标
y1	unsigned int, 线条起始坐标中的 y 坐标
x2	unsigned int, 线条终点坐标中的 x 坐标
y2	unsigned int, 线条终点坐标中的 y 坐标
line_width	unsigned int, 线条宽度
line_color	IxOSD_ColorParams, 线条颜色

转换函数

将给定的对象或数据转换为 IxOSD_LineParams 格式,调用方式为 pyixstream.IxOSD_LineParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.13 pyixstream.IxOSD_ArrowParams

描述

(结构体) 在图像中绘制箭头时的参数

数据字段

x1	unsigned int, 箭头起始坐标的 x 坐标
y1	unsigned int, 箭头起始坐标的 y 坐标

x2	unsigned int, 箭头终止坐标的 x 坐标
y2	unsigned int, 箭头终止坐标的 y 坐标
arrow_width	unsigned int, 箭头的宽度
arrow_head	IxOSD_Arrow_Head_Direction, 箭头的方向
arrow_color	IxOSD_ColorParams, 箭头的颜色
reserved	unsigned int, 保留字段

转换函数

将给定的对象或数据转换为 IxOSD_ArrowParams 格式,调用方式为 pyixstream.IxOSD_ArrowParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.14 pyixstream.IxOSD_CircleParams

描述

(结构体) 在图像中绘制圆形时的参数

数据字段

xc	unsigned int, 圆心坐标的 x 坐标
yc	unsigned int, 圆心坐标的 y 坐标
radius	unsigned int, 圆的半径
circle_color	IxOSD_ColorParams, 圆边框的颜色
has_bg_color	unsigned int, 是否对圆进行填充的标志
bg_color	IxOSD_ColorParams, 圆的填充色
reserved	unsigned int, 保留字段

转换函数

将给定的对象或数据转换为 IxOSD_CircleParams 格式,调用方式为 pyixstream.IxOSD_CircleParams.cast(data)

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.15 pyixstream.IxBaseMeta

描述

(结构体) 基础元数据

数据字段

batch_meta	struct_IxBatchMeta *batch 元数据
meta_type	IxMetaType, 元数据类型
uContext	void *, 上下文

转换函数

将给定的对象或数据转换为 IxBaseMeta 格式, 调用方式为 pyixstream.IxBaseMeta.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.16 pyixstream.IxUserMeta

描述

(结构体) 用户元数据

数据字段

base_meta	IxBaseMeta, 基础元数据
user_meta_data	gpointer, 用户元数据

转换函数

将给定的对象或数据转换为 IxUserMeta 格式, 调用方式为 pyixstream.IxUserMeta.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.17 pyixstream.IxLabelInfo

描述

(结构体) 标签的元数据

数据字段

base_meta	IxBaseMeta, 基础元数据
num_classes	guint, 给定标签的数量
result_label	str, 标签数据
pResult_label	str, 标签数据
result_class_id	guint, 标签的 ID
label_id	guint, 多个分类器时保存标签 ID
result_prob	gfloat, 分类结果的可能性

转换函数

将给定的对象或数据转换为 IxLabelInfo 格式, 调用方式为 `pyxistream.IxLabelInfo.cast(data)`

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.18 pyxistream.IxClassifierMeta

描述

(结构体) 分类器的元数据

数据字段

base_meta	IxBaseMeta, 基础元数据
num_labels	guint, 目标的 label 数量
unique_component_id	gint, 分类器元数据的 ID
label_info_list	IxLabelInfoList list, 标签数据列表

转换函数

将给定的对象或数据转换为 IxClassifierMeta 格式, 调用方式为 `pyxistream.IxClassifierMeta.cast(data)`

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.19 pyxistream.IxDisplayMeta

描述

(结构体) 显示的元数据 (用户需要在图像上绘制时, 可以通过该结构体添加参数)

数据字段

base_meta	IxBaseMeta, 基础元数据
num_rects	guint, 矩形框的数量
num_labels	guint, 文本的数量
num_lines	guint, 线条的数量
num_arrows	guint, 箭头的数量
num_circles	guint, 圆的数量
num_polygons	guint, 多边形的数量
rect_params	IxOSD_TextParams list, 矩形框的参数信息
line_params	IxOSD_LineParams list, 线条的参数信息
arrow_params	IxOSD_ArrowParams list, 箭头的参数信息
circle_params	IxOSD_CircleParams list, 圆的参数信息
polygon_params	IxOSD_PolygonParams list, 多边形的参数信息
reserved	gint64 list, 保留字段

转换函数

将给定的对象或数据转换为 IxDisplayMeta 格式, 调用方式为 pyixstream.IxDisplayMeta.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.20 pyixstream.IxObjectMeta

描述

(结构体) 目标的元数据

数据字段

base_meta	IxBaseMeta, 基础元数据
parent	struct_IxObjectMeta *, 目标元数据的父节点
unique_component_id	gint, 目标元数据的 ID
class_id	gint, 目标的类别 ID
object_id	guint64, 目标经过跟踪后的 ID
detector_bbox_info	Ix_BboxInfo, 目标的检测框信息

tracker_bbox_info	Ix_BboxInfo, 目标经过跟踪后的检测框信息
confidence	gfloat, 目标的置信度
rect_params	IxOSD_RectParams, 目标的边框参数
mask_params	IxOSD_MaskParams, 目标的 mask 参数
text_params	IxOSD_TextParams, 目标的文本参数
obj_label	str, 目标的标签信息
classifier_meta_list	IxClassifierMetaList list, 目标元数据中的分类器元数据列表
obj_user_meta_list	IxUserMetaList list, 目标元数据中的用户元数据列表
reserved	gint64 list, 保留字段

转换函数

将给定的对象或数据转换为 IxObjectMeta 格式, 调用方式为 `pyxistream.IxObjectMeta.cast(data)`

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.21 pyxistream.IxFrameMeta

描述

(结构体) 帧的元数据

数据字段

base_meta	IxBaseMeta, 基础元数据
pad_index	guint, 当前帧在 batch 中的编号
batch_id	guint, 当前帧所在 batch 的编号
frame_num	guint, 当前帧在数据流中的编号
buf_pts	guint64, 当前帧的 pts 时间戳
ntp_timestamp	guint64, 当前帧的 ntp 时间戳
source_id	guint, 当前帧在 batch 中的编号
source_frame_width	guint, 输入数据的宽度
source_frame_height	guint, 输入数据的高度
num_obj_meta	guint, 目标元数据的数量
bInferDone	gboolean, 帧数据是否推理的标志位

obj_meta_list	IXObjectMetaList list, 帧内目标的元数据列表
display_meta_list	IXDisplayMetaList list, 帧内部的显示元数据列表
frame_user_meta_list	IXUserMetaList list, 帧内部的用户元数据列表
pipeline_width	guint, 多路合并后输出的图像宽度
pipeline_height	guint, 多路合并后输出的图像高度
reserved	gint64 list, 保留字段

转换函数

将给定的对象或数据转换为 IxFrameMeta 格式, 调用方式为 pyixstream.IxFrameMeta.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.22 pyixstream.IxBatchMeta

描述

(结构体) batch 元数据

数据字段

base_meta	IXBaseMeta, 基础元数据
max_frames_in_batch	guint, batch 中最大的帧数
num_frames_in_batch	guint, batch 中的帧数
frame_meta_list	IXFrameMetaList list, 帧元数据的内存池
batch_user_meta_list	IXUserMetaList list, batch 中用户元数据的列表
meta_mutex	GRecMutex, 元数据锁

转换函数

将给定的对象或数据转换为 IxBatchMeta 格式, 调用方式为 pyixstream.IxBatchMeta.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.23 pyixstream.IxInferSegmentationMeta

描述

(结构体) 分割网络输出元数据

数据字段

classes	guint, 分割网络输出的类别个数
width	guint, 分割网络输出类别图的宽度
height	guint, 分割网络输出类别图的高度
class_map	guint array, 分割网络输出类别图
class_probabilities_map	gfloat array, 分割网络输出的每种类别的概率数据
priv_data	void *, 私有数据

转换函数

将给定的对象或数据转换为 IxInferSegmentationMeta 格式,调用方式为 pyixstream.IxInferSegmentationMeta.cast(data)

```
1,cast(self: capsule)
```

1.7.1.24 pyixstream.IxAnalyticsObjInfo

描述

(结构体) 分析插件输出的目标信息

数据字段

roiStatus	str list, 保存目标在 ROI 区域的标签信息
ocStatus	str list, 保存目标在 ROI 区域的过度拥挤标签信息
lcStatus	str list, 保存目标的越线信息
dirStatus	str list, 保存目标的方向信息
unique_id	guint, 元数据的唯一标识符

转换函数

将给定的对象或数据转换为 IxAnalyticsObjInfo 格式,调用方式为 pyixstream.IxAnalyticsObjInfo.cast(data)

```
1,cast(self: capsule)
```

1.7.1.25 pyixstream.IxAnalyticsFrameMeta

描述

(结构体) 分析插件输出的帧的元数据

数据字段

ocStatus	dict<str,bool>, 当前帧分析插件输出的过度拥挤状态信息
objInROIcnt	dict<str,int>, 保存当前帧在 ROI 区域的目标数量
objLCCurrCnt	dict<str,int>, 保存当前帧越线目标数量和标签
objLCCumCnt	dict<str,int>, 保存累积到当前帧的越线目标总数和标签
unique_id	guint, 元数据的唯一标识符
objCnt	dict<str,int>, 当前帧每种类别目标的总数

转换函数

将给定的对象或数据转换为 IxAnalyticsFrameMeta 格式,调用方式为 `pyxistream.IxAnalyticsFrameMeta.cast(data)`

```
1,cast(self: capsule)
```

1.7.1.26 pyxistream.IxInferDims

描述

(结构体) 推理网络的尺寸信息

数据字段

numDims	guint, 网络层的维度数量
d	guint array, 网络层每一个维度的尺寸
numElements	guint, 网络层所有维度的尺寸和

转换函数

将给定的对象或数据转换为 IxInferDims 格式,调用方式为 `pyxistream.IxInferDims.cast(data)`

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.1.27 pyxistream.IxInferDimsCHW

描述

(结构体) 推理网络的尺寸信息 (数据排布为 CHW)

数据字段

c	gint, 网络层的通道数量
h	gint, 网络层的高度
w	gint, 网络层的宽度

转换函数

将给定的对象或数据转换为 IxInferDimsCHW 格式, 调用方式为 `pyxstream.IxInferDimsCHW.cast(data)`

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.28 pyxstream.IxInferDataType

描述

(结构体) 网络层的数据类型

数据字段

FLOAT	0 浮点型
HALF	1 半精度浮点
INT8	2 字符型
INT32	3 整型

1.7.1.29 pyxstream.IxInferLayerInfo

描述

(结构体) 推理网络的层信息

数据字段

dataType	IxInferDataType, 网络层的数据类型
inferDims	IxInferDims, 网络层的维度信息
bindingIndex	int, 网络层的索引

layerName	str, 网络层的名称
buffer	void *, 网络层的数据
isInput	int, 当前层是否为输入层的标志

转换函数

将给定的对象或数据转换为 IxInferLayerInfo 格式, 调用方式为 pyixstream.IxInferLayerInfo.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.30 pyixstream.IxInferNetworkInfo

描述

(结构体) 推理网络的信息

数据字段

width	guint, 网络输入的宽度
height	guint, 网络层的高度
channels	guint, 网络层的通道数量

转换函数

将给定的对象或数据转换为 IxInferNetworkInfo 格式, 调用方式为 pyixstream.IxInferNetworkInfo.cast(data)

```
1, cast(self: capsule)
2, cast(self: int)
```

1.7.1.31 pyixstream.IxInferObjectDetectionInfo

描述

(结构体) 检测目标的相关信息

数据字段

classId	guint, 目标的类别 ID
left	gfloat, 目标检测框左上顶点在水平方向的偏移

top	gfloat, 目标检测框左上顶点在垂直方向的偏移
width	gfloat, 目标检测框的宽度
height	gfloat, 目标检测框的高度
detectionConfidence	gfloat, 目标的置信度

转换函数

将给定的对象或数据转换为 IxInferObjectDetectionInfo 格式,调用方式为 pyixstream.IxInferObjectDetectionInfo.cast(da

```
1,cast(self: capsule)
2,cast(self: int)
```

1.7.2 函数接口

1.7.2.1 pyixstream.ix_acquire_frame_meta_from_pool

功能概述

从内存池中获取帧的元数据的内存

函数原型

```
pyixstream.ix_acquire_frame_meta_from_pool(batch_meta: pyixstream.IxBatchMeta)->
↳ pyixstream.IxFrameMeta
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxFrameMeta 类型的对象

1.7.2.2 pyixstream.ix_add_frame_meta_to_batch

功能概述

将帧的元数据添加到 batch 的元数据中

函数原型

```
pyixstream.ix_add_frame_meta_to_batch(batch_meta: pyixstream.IxBatchMeta, frame_meta:
↳ pyixstream.IxFrameMeta)-> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象
- frame_meta: 需要添加的 IxFrameMeta 类型的元数据对象

返回值

无

1.7.2.3 pyixstream.ix_remove_frame_meta_from_batch

功能概述

从 batch 的元数据中移除帧的元数据

函数原型

```
pyixstream.ix_remove_frame_meta_from_batch(batch_meta: pyixstream.IxBatchMeta,  
↳ frame_meta: pyixstream.IxFrameMeta) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象
- frame_meta: 需要移除的 IxFrameMeta 类型的元数据对象

返回值

无

1.7.2.4 pyixstream.ix_acquire_obj_meta_from_pool

功能概述

从内存池中获取目标的元数据内存

函数原型

```
pyixstream.ix_acquire_obj_meta_from_pool(batch_meta: pyixstream.IxBatchMeta) ->  
↳ pyixstream.IxObjectMeta
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxObjectMeta 类型的对象

1.7.2.5 pyixstream.ix_add_obj_meta_to_frame

功能概述

将目标的元数据添加到帧的元数据中

函数原型

```
pyixstream.ix_add_obj_meta_to_frame(frame_meta: pyixstream.IxFrameMeta, obj_meta:  
↳ pyixstream.IxObjectMeta, obj_parent: pyixstream.IxObjectMeta)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- obj_meta: 需要添加的 IxObjectMeta 类型的元数据对象
- obj_parent: IxObjectMeta 类型的元数据的父节点对象，可以为空

返回值

无

1.7.2.6 pyixstream.ix_remove_obj_meta_from_frame

功能概述

从帧的元数据中移除目标的元数据

函数原型

```
pyixstream.ix_remove_obj_meta_from_frame(frame_meta: pyixstream.IxFrameMeta, obj_meta:  
↳ pyixstream.IxObjectMeta)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- obj_meta: 需要移除的 IxObjectMeta 类型的元数据对象

返回值

无

1.7.2.7 pyixstream.ix_acquire_classifier_meta_from_pool

功能概述

从内存池中获取分类器的元数据内存

函数原型

```
pyixstream.ix_acquire_classifier_meta_from_pool(batch_meta: pyixstream.IxBatchMeta)->  
↳ pyixstream.IxClassifierMeta
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxClassifierMeta 类型的对象

1.7.2.8 pyixstream.ix_add_classifier_meta_to_obj

功能概述

将分类器的元数据添加到目标的元数据中

函数原型

```
pyixstream.ix_add_classifier_meta_to_obj(obj_meta: pyixstream.IXObjectMeta,  
↪ classifier_meta: pyixstream.IxClassifierMeta) -> None
```

参数说明

- obj_meta: IXObjectMeta 类型的元数据对象
- classifier_meta: IxClassifierMeta 类型的需要添加的元数据对象

返回值

无

1.7.2.9 pyixstream.ix_remove_classifier_meta_from_obj

功能概述

从目标的元数据中移除分类器的元数据

函数原型

```
pyixstream.ix_remove_classifier_meta_from_obj(obj_meta: pyixstream.IXObjectMeta ,  
↪ classifier_meta: pyixstream.IxClassifierMeta ) -> None
```

参数说明

- obj_meta: IXObjectMeta 类型的元数据对象
- classifier_meta: IxClassifierMeta 类型的需要移除的元数据对象

返回值

无

1.7.2.10 pyixstream.ix_acquire_display_meta_from_pool

功能概述

从内存池中获取显示元数据的内存

函数原型

```
pyixstream.ix_acquire_display_meta_from_pool(batch_meta: pyixstream.IxBatchMeta) ->  
↪ pyixstream.IxDisplayMeta
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxDisplayMeta 类型的对象

1.7.2.11 pyixstream.ix_add_display_meta_to_frame

功能概述

将需要显示的元数据添加到帧的元数据中

函数原型

```
pyixstream.ix_add_display_meta_to_frame(frame_meta: pyixstream.IxFrameMeta, display_meta:  
↳ pyixstream.IxDisplayMeta) -> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- display_meta: IxDisplayMeta 类型的元数据对象

返回值

无

1.7.2.12 pyixstream.ix_remove_display_meta_from_frame

功能概述

从帧的元数据中移除要显示的元数据

函数原型

```
pyixstream.ix_remove_display_meta_from_frame(frame_meta: pyixstream.IxFrameMeta,  
↳ display_meta: pyixstream.IxDisplayMeta) -> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- display_meta: 需要移除的 IxDisplayMeta 类型的元数据对象

返回值

无

1.7.2.13 pyixstream.ix_acquire_label_info_meta_from_pool

功能概述

从内存池中获取标签元数据的内存

函数原型

```
pyxistream.ix_acquire_label_info_meta_from_pool(batch_meta: pyxistream.IxBatchMeta) ->  
↳ pyxistream.IxLabelInfo
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxLabelInfo 类型的对象

1.7.2.14 pyxistream.ix_add_label_info_meta_to_classifier

功能概述

将标签数据添加到分类器的元数据中

函数原型

```
pyxistream.ix_add_label_info_meta_to_classifier(classifier_meta:  
↳ pyxistream.IxClassifierMeta, label_info_meta: pyxistream.IxLabelInfo) -> None
```

参数说明

- classifier_meta: IxClassifierMeta 类型的元数据对象
- label_info_meta: 需要添加 IxLabelInfo 类型的数据对象

返回值

无

1.7.2.15 pyxistream.ix_remove_label_info_meta_from_classifier

功能概述

从分类器的元数据中移除标签元数据

函数原型

```
pyxistream.ix_remove_label_info_meta_from_classifier(classifier_meta:  
↳ pyxistream.IxClassifierMeta, label_info_meta: pyxistream.IxLabelInfo) -> None
```

参数说明

- classifier_meta: IxClassifierMeta 类型的元数据对象
- label_info_meta: 需要移除 IxLabelInfo 类型的数据对象

返回值

无

1.7.2.16 pyixstream.ix_acquire_user_meta_from_pool

功能概述

从内存池中获取用户元数据的内存

函数原型

```
pyixstream.ix_acquire_user_meta_from_pool(batch_meta: pyixstream.IxBatchMeta) ->  
↳ pyixstream.IxUserMeta
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

- IxUserMeta 类型的对象

1.7.2.17 pyixstream.ix_add_user_meta_to_batch

功能概述

将用户元数据添加到 batch 的元数据中

函数原型

```
pyixstream.ix_add_user_meta_to_batch(batch_meta: pyixstream.IxBatchMeta,  
↳ user_meta: pyixstream.IxUserMeta) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象
- user_meta: 需要添加 IxUserMeta 类型的元数据对象

返回值

无

1.7.2.18 pyixstream.ix_remove_user_meta_from_batch

功能概述

从 batch 的元数据中移除用户元数据

函数原型

```
pyixstream.ix_remove_user_meta_from_batch(batch_meta: pyixstream.IxBatchMeta,  
↳ user_meta: pyixstream.IxUserMeta) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象
- user_meta: 需要移除 IxUserMeta 类型的元数据对象

返回值

无

1.7.2.19 pyixstream.ix_add_user_meta_to_frame

功能概述

将用户元数据添加到帧的元数据中

函数原型

```
pyixstream.ix_add_user_meta_to_frame(frame_meta: pyixstream.IxFrameMeta,  
↪ user_meta:pyixstream.IxUserMeta)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- user_meta: 需要添加 IxUserMeta 类型的元数据对象

返回值

无

1.7.2.20 pyixstream.ix_remove_user_meta_from_frame

功能概述

从帧的元数据中移除用户元数据

函数原型

```
pyixstream.ix_remove_user_meta_from_frame(frame_meta: pyixstream.IxFrameMeta,  
↪ user_meta:pyixstream.IxUserMeta)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- user_meta: 需要移除 IxUserMeta 类型的元数据对象

返回值

无

1.7.2.21 pyixstream.ix_add_user_meta_to_obj

功能概述

将用户元数据添加到目标的元数据中

函数原型

```
pyixstream.ix_add_user_meta_to_obj(obj_meta: IxObjectMeta.IxFrameMeta,  
↪ user_meta:pyixstream.IxUserMeta)-> None
```

参数说明

- obj_meta: IxObjectMeta 类型的元数据对象
- user_meta: 需要添加的 IxUserMeta 类型的用户元数据对象

返回值

无

1.7.2.22 pyixstream.ix_remove_user_meta_from_obj

功能概述

从目标的元数据内存中移除用户元数据

函数原型

```
pyixstream.ix_remove_user_meta_from_obj(obj_meta: IxObjectMeta.IxFrameMeta,  
↪ user_meta:pyixstream.IxUserMeta)-> None
```

参数说明

- obj_meta: IxObjectMeta 类型的元数据对象
- user_meta: 需要移除的 IxUserMeta 类型的用户元数据对象

返回值

无

1.7.2.23 pyixstream.ix_get_nth_frame_meta

功能概述

从帧的元数据列表中获取第 n 帧的元数据

函数原型

```
pyixstream.ix_get_nth_frame_meta(frame_meta_list: pyixstream.GList, index: int)->  
↪ pyixstream.IxFrameMeta
```

参数说明

- frame_meta_list: 帧的元数据列表指针
- index: 需要获取的元数据的序号

返回值

- IxFrameMeta 类型的指针

1.7.2.24 pyixstream.ix_clear_frame_meta_list

功能概述

从 batch 的元数据中删除帧的元数据列表

函数原型

```
pyixstream.ix_clear_frame_meta_list(batch_meta: pyixstream.IxBatchMeta, meta_list:  
↪ pyixstream.GList) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据指针
- meta_list: 需要删除的帧的元数据列表

返回值

无

1.7.2.25 pyixstream.ix_clear_obj_meta_list

功能概述

从帧的元数据中删除目标的元数据列表

函数原型

```
pyixstream.ix_clear_obj_meta_list(frame_meta: pyixstream.IxFrameMeta, meta_list:  
↪ pyixstream.GList) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据指针
- meta_list: 需要删除的目标元数据列表

返回值

无

1.7.2.26 pyixstream.ix_clear_classifier_meta_list

功能概述

从目标元数据中删除分类器的元数据列表

函数原型

```
pyixstream.ix_clear_classifier_meta_list(obj_meta: pyixstream.IxObjectMeta, meta_list:  
↪ pyixstream.GList)-> None
```

参数说明

- obj_meta: IxObjectMeta 类型的元数据对象
- meta_list: 需要删除的分类器元数据列表

返回值

无

1.7.2.27 pyixstream.ix_clear_label_info_meta_list

功能概述

从分类器元数据中删除标签列表

函数原型

```
pyixstream.ix_clear_label_info_meta_list(classifier_meta: pyixstream.IxClassifierMeta,  
↪ meta_list: pyixstream.GList)-> None
```

参数说明

- classifier_meta: IxClassifierMeta 类型的元数据对象
- meta_list: 需要删除的标签列表

返回值

无

1.7.2.28 pyixstream.ix_clear_display_meta_list

功能概述

从帧的元数据中删除显示元数据

函数原型

```
pyixstream.ix_clear_display_meta_list(frame_meta: pyixstream.IxFrameMeta, meta_list:  
↪ pyixstream.GList)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- meta_list: 需要删除的显示元数据列表

返回值

无

1.7.2.29 pyixstream.ix_clear_batch_user_meta_list

功能概述

从 batch 的元数据中删除用户元数据列表

函数原型

```
pyixstream.ix_clear_batch_user_meta_list(batch_meta: pyixstream.IxBatchMeta, meta_list:
↳ pyixstream.GList)-> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象
- meta_list: 需要删除的用户元数据列表

返回值

无

1.7.2.30 pyixstream.ix_clear_frame_user_meta_list

功能概述

从帧的元数据中删除用户元数据列表

函数原型

```
pyixstream.ix_clear_frame_user_meta_list(frame_meta: pyixstream.IxFrameMeta, meta_list:
↳ pyixstream.GList)-> None
```

参数说明

- frame_meta: IxFrameMeta 类型的元数据对象
- meta_list: 需要删除的用户元数据列表

返回值

无

1.7.2.31 pyixstream.ix_get_user_meta_type

功能概述

获取用户元数据的类型

函数原型

```
pyixstream.ix_get_user_meta_type(meta_descriptor: str)-> int
```

参数说明

- meta_descriptor: 用户元数据的字符串描述符

返回值

- IxMetaType 的枚举值

1.7.2.32 pyixstream.gst_buffer_add_ix_meta

功能概述

在内存中添加元数据

函数原型

```
pyixstream.gst_buffer_add_ix_meta(buffer: GstBuffer, meta_data: capsule, user_data:  
↳ capsule, copy_func: void* (void*, void*), release_func: void (void*, void*))->  
↳ pyixstream.IxMeta
```

参数说明

- buffer: 需要添加元数据的 GstBuffer 对象
- meta_data: 需要添加的元数据对象
- user_data: 需要添加的用户数据
- copy_func: IxMetaCopyFunc 类型函数, 用来拷贝 meta 数据的拷贝函数
- release_func: IxMetaReleaseFunc 类型函数用来释放 meta 数据的函数

返回值

- IxMeta 类型的对象

1.7.2.33 pyixstream.gst_buffer_get_ix_meta

功能概述

从内存中获取元数据

函数原型

```
pyxistream.gst_buffer_get_ix_meta(buffer: GstBuffer) -> pyxistream.IxMeta
```

参数说明

- buffer: 获取元数据的 GstBuffer 对象

返回值

- IxMeta 类型的对象

1.7.2.34 pyxistream.gst_buffer_get_ix_batch_meta

功能概述

从内存中获取 batch 的元数据指针

函数原型

```
pyxistream.gst_buffer_get_ix_batch_meta(buffer: int) -> pyxistream.IxBatchMeta
```

参数说明

- buffer: 获取元数据的内存对象

返回值

- IxBatchMeta 类型的对象

1.7.2.35 pyxistream.ix_acquire_meta_lock

功能概述

获取锁

函数原型

```
pyxistream.ix_acquire_meta_lock(batch_meta: pyxistream.IxBatchMeta) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

无

1.7.2.36 pyixstream.ix_release_meta_lock

功能概述

释放锁

函数原型

```
pyixstream.ix_release_meta_lock(batch_meta: pyixstream.IxBatchMeta) -> None
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

无

1.7.2.37 pyixstream.ix_create_batch_meta

功能概述

创建 IxBatchMeta 类型的元数据

函数原型

```
pyixstream.ix_create_batch_meta(max_batch_size: int) -> pyixstream.IxBatchMeta
```

参数说明

- max_batch_size: 创建的批处理大小

返回值

- IxBatchMeta 类型的对象

1.7.2.38 pyixstream.ix_destory_batch_meta

功能概述

释放创建的 IxBatchMeta 类型的元数据

函数原型

```
pyixstream.ix_destory_batch_meta(batch_meta: pyixstream.IxBatchMeta) -> int
```

参数说明

- batch_meta: IxBatchMeta 类型的元数据对象

返回值

无

1.7.2.39 pyixstream.get_ix_buf_surface

功能概述

获取帧数据 (返回的 ndarray 中为依 IDd 获取的当前帧的数据信息)

函数原型

```
pyixstream.get_ix_buf_surface(gst_buffer: int, batchID: int) -> numpy.ndarray
```

参数说明

- gst_buffer: 内存对象
- batchID: ID

返回值

- numpy.ndarray 类型的对象

1.7.2.40 pyixstream.get_ix_buf_surface_gpu

功能概述

获取帧数据 (返回的 tuple 中为依据 ID 获取的当前帧的数据信息, 包含图像的数据类型, 数据 shape, 大小, 以及图像数据)

函数原型

```
pyixstream.get_ix_buf_surface_gpu(gst_buffer: int, batchID: int) -> tuple
```

参数说明

- gst_buffer: 内存对象
- batchID: ID

返回值

- tuple 类型的对象

1.7.2.41 pyixstream.get_segmentation_masks

功能概述

获取分割网络的元数据

函数原型

```
pyixstream.get_segmentation_masks(data: capsule) -> numpy.ndarray
```

参数说明

- data: IxInferSegmentationMeta 类型的数据对象

返回值

- numpy.ndarray 类型的对象

1.7.2.42 pyixstream.get_ix_LayerInfo

功能概述

获取网络的输出层信息

函数原型

```
pyixstream.get_ix_LayerInfo(data: capsule) -> pyixstream.IxInferLayerInfo
```

参数说明

- data: IxInferTensorMeta 类型的数据对象

返回值

- IxInferLayerInfo 类型的对象

1.7.2.43 pyixstream.user_copyfunc

功能概述

为元数据设置 copy 函数

函数原型

```
pyixstream.user_copyfunc(meta: pyixstream.IxUserMeta, func: Callable[[capsule, capsule],  
↪ capsule]) -> None
```

参数说明

- meta: 需要设置 copy 函数的 IxUserMeta 对象
- func: 回调函数

返回值

无

1.7.2.44 pyixstream.user_releasefunc

功能概述

为元数据设置 release 函数

函数原型

```
pyixstream.user_releasefunc(meta: pyixstream.IxUserMeta, func: Callable[[capsule,  
↪ capsule], capsule]) -> None
```

参数说明

- meta: 需要设置 release 函数的 IxUserMeta 对象

- func: 回调函数

返回值

无

1.7.2.45 pyixstream.alloc_buffer

功能概述

根据尺寸开辟内存

函数原型

```
pyixstream.alloc_buffer(size: int) -> int
```

参数说明

- size: 需要开辟的内存大小

返回值

- 开辟内存的地址

1.7.2.46 pyixstream.free_buffer

功能概述

根据地址释放内存

函数原型

```
pyixstream.free_buffer(buffer: int) -> None
```

参数说明

- buffer: 需要释放的内存地址

返回值

无

1.7.2.47 pyixstream.free_gbuffer

功能概述

根据指针释放内存

函数原型

```
pyxistream.free_gbuffer(buffer: capsule)-> None
```

参数说明

- buffer: 需要释放的内存

返回值

无

1.7.2.48 pyxistream.get_string

功能概述

将给定的指针转换为字符串

函数原型

```
pyxistream.get_string(ptr: int)-> str
```

参数说明

- ptr: 给定的指针

返回值

- 转换后的 str

1.7.2.49 pyxistream.get_ptr

功能概述

获取给定对象的 C 地址

函数原型

```
pyxistream.get_ptr(ptr: capsule)-> int
```

参数说明

- ptr: 给定的对象

返回值

- 返回的地址

1.8 IxStream C++ API 参考

1.8.1 数据结构

1.8.1.1 IxBufSurfaceColorFormat

描述

(枚举) 图像格式的枚举值

数据字段

IXBUF_COLOR_FORMAT_INVALID	0	图像格式无效
IXBUF_COLOR_FORMAT_YUV420	1	图像格式为 YUV420
IXBUF_COLOR_FORMAT_NV12	2	图像格式为 NV12
IXBUF_COLOR_FORMAT_NV21	3	图像格式为 NV21
IXBUF_COLOR_FORMAT_RGB	4	图像格式为 RGB(NCHW)
IXBUF_COLOR_FORMAT_BGR	5	图像格式为 BGR(NCHW)
IXBUF_COLOR_FORMAT_RGBx	6	图像格式为 RGBx(NCHW)
IXBUF_COLOR_FORMAT_BGRx	7	图像格式为 BGRx(NCHW)
IXBUF_COLOR_FORMAT_R32F_G32F_B32F	8	图像格式为 RGB 类型的 float 数据 (NCHW)
IXBUF_COLOR_FORMAT_GRAY8	9	图像格式为 GRAY
IXBUF_COLOR_FORMAT_GRAY32F	10	图像格式为 GRAY 类型的 float 数据
IXBUF_COLOR_FORMAT_LAST	11	无具体指向, 最后一个枚举值

1.8.1.2 IxBufSurfaceMemType

描述

(枚举) 内存类型

数据字段

IXBUF_MEM_DEFAULT	0 内存类型为默认值 (IXBUF_MEM_CUDA_DEVICE)
IXBUF_MEM_CUDA_PINNED	1 内存类型为 GPU 锁页内存
IXBUF_MEM_CUDA_DEVICE	2 内存类型为 GPU 内存
IXBUF_MEM_CUDA_UNIFIED	3 内存类型为 GPU 统一内存
IXBUF_MEM_SYSTEM	4 内存类型为 CPU 内存

1.8.1.3 IxBufSurfacePlaneParams

描述

(枚举) 图像的平面参数 (图像的通道参数)

数据字段

uint32_t num_planes	平面的数量
uint32_t width[IXBUF_MAX_PLANES]	用来保存不同平面的宽度的数组
uint32_t height[IXBUF_MAX_PLANES]	用来保存不同平面的高度的数组
uint32_t pitch[IXBUF_MAX_PLANES]	用来保存不同平面的对齐宽度的数组
uint32_t offset[IXBUF_MAX_PLANES]	用来保存不同平面的偏移的数组
uint32_t psize[IXBUF_MAX_PLANES]	用来保存不同平面的大小的数组
uint32_t bytesPerPix[IXBUF_MAX_PLANES]	用来保存不同平面像素所占字节数的数组
void * _reserved	保留字段

1.8.1.4 IxBufSurfaceParams

描述

(结构体) 保存 batch 内存中单个图像的内存信息

数据字段

guint width	图像的宽度
uint height	图像的高度
IxBufSurfaceColorFormat colorFormat	图像的格式
guint dataSize	图像的大小
void *dataPtr	图像存储的内存指针
IxBufSurfacePlaneParams planeParams	图像的平面参数
void *_reserved	保留字段

1.8.1.5 IxBufSurface

描述

(结构体) 保存 batch 内存的信息

数据字段

guint gpuId	batch 内存所在的 GPU 编号
guint batchSize	batch 的大小
guint numFilled	batch 内存中实际填充的内存数量
gboolean isContiguous	batch 内存中所有的内存是否连续的标志位
IxBufSurfaceMemType memType	batch 内存的内存类型
IxBufSurfaceParams *surfaceList	batch 内存中存储的图像列表
void *_reserved	保留字段

1.8.1.6 IxMetaType

描述

(枚举) 元数据类型

数据字段

IX_INVALID_META	无效的元数据
IX_BATCH_META	batch 元数据
IX_FRAME_META	帧元数据
IX_OBJ_META	目标元数据
IX_DISPLAY_META	显示元数据
IX_CLASSIFIER_META	分类器元数据
IX_LABEL_INFO_META	标签元数据
IX_USER_META	用户元数据
IX_PAYLOAD_META	负荷元数据
IX_EVENT_MSG_META	事件消息元数据
IX_OPTICAL_FLOW_META	光流元数据
IX_LATENCY_MEASUREMENT_META	延迟测量元数据
IXINFER_TENSOR_OUTPUT_META	推理张量输出元数据
IXINFER_SEGMENTATION_META	分割网络元数据
IX_CROP_IMAGE_META	编码目标的裁剪元数据
IX_TRACKER_PAST_FRAME_META	跟踪过去帧设置的元数据
IX_AUDIO_BATCH_META	音频 batch 元数据
IX_AUDIO_FRAME_META	音频帧元数据
IX_PREPROCESS_FRAME_META	指定预处理帧元数据
IX_PREPROCESS_BATCH_META	指定预处理 batch 元数据
IX_CUSTOM_MSG_BLOB	经典信息块的元数据
IX_RESERVED_META	保留元数据
IX_GST_CUSTOM_META	指定枚举类型的经典值
IX_START_USER_META	用户元数据的起始值
IX_PREPROCESS_BATCH_OBJ_META	指定预处理 batch 的目标元数据

1.8.1.7 IxOSD_Arrow_Head_Direction

描述

(枚举) 在图像中绘制箭头时的方向值

数据字段

START_HEAD	0 箭头在坐标的起始处
END_HEAD	1 箭头坐标的终止处
BOTH_HEAD	2 双向箭头

1.8.1.8 IxOSD_ColorParams

描述

(结构体) 在图像中绘制时绘制对象的颜色参数 (颜色参数为 RGBA，值的范围为 0.0 至 1.0)

数据字段

double red	色域空间中的 R 分量
double green	色域空间中的 G 分量
double blue	色域空间中的 B 分量
double alpha	色域空间中的 A 分量

1.8.1.9 IxOSD_FontParams

描述

(结构体) 在图像中绘制文本时文本的字体参数

数据字段

char *font_name	字体名称
unsigned int font_size	字体大小
IxOSD_ColorParams font_color	字体颜色

1.8.1.10 IxOSD_TextParams

描述

(结构体) 在图像中绘制文本时文本的参数

数据字段

char *display_text	绘制的文本
--------------------	-------

unsigned int x_offset	文本所在位置的 x 坐标
unsigned int y_offset	文本所在位置的 y 坐标
IxOSD_FontParams font_params	文本的字体参数
int set_bg_clr	设置背景颜色标志
IxOSD_ColorParams text_bg_clr	背景颜色

1.8.1.11 IxOSD_RectParams

描述

(结构体) 在图像中绘制矩形框时的矩形框参数

数据字段

float left	矩形框左上顶点的 x 坐标
float top	矩形框左上顶点的 y 坐标
float width	矩形框的宽度
float height	矩形框的高度
unsigned int border_width	矩形框边框线的宽度
IxOSD_ColorParams border_color	矩形框边框线的颜色
unsigned int has_bg_color	矩形框填充标志
unsigned int reserved	保留字段
IxOSD_ColorParams bg_color	矩形框的填充颜色

1.8.1.12 IxOSD_MaskParams

描述

(结构体) 在图像中绘制 mask 时的参数

数据字段

float *data	mask 内存
unsigned int size	mask 内存的大小
float threshold	阈值
unsigned int width	mask 的宽度
unsigned int height	mask 的高度

1.8.1.13 IxOSD_LineParams

描述

(结构体) 在图像中绘制线条时的参数

数据字段

unsigned int x1	线条起始坐标中的 x 坐标
unsigned int y1	线条起始坐标中的 y 坐标
unsigned int x2	线条终点坐标中的 x 坐标
unsigned int y2	线条终点坐标中的 y 坐标
unsigned int line_width	线条宽度
IxOSD_ColorParams line_color	线条颜色

1.8.1.14 IxOSD_PolygonParams

描述

(结构体) 在图像中绘制多边形时的参数

数据字段

int point_2d[512]	保存多边形的各顶点坐标 (按照 x1, y1, x2, y2... 的顺序排列)
unsigned int num_pts	多边形的顶点个数
unsigned int border_width	多边形的线条宽度
bool isClosed	多边形是否闭合标志位
IxOSD_ColorParams border_color	多边形边框的颜色
unsigned int has_bg_color	是否对多边形进行填充标志
IxOSD_ColorParams bg_color	多边形填充的颜色

1.8.1.15 IxOSD_ArrowParams

描述

(结构体) 在图像中绘制箭头时的参数

数据字段

unsigned int x1	箭头起始坐标的 x 坐标
-----------------	--------------

unsigned int y1	箭头起始坐标的 y 坐标
unsigned int x2	箭头终止坐标的 x 坐标
unsigned int y2	箭头终止坐标的 y 坐标
unsigned int arrow_width	箭头的宽度
IxOSD_Arrow_Head_Direction arrow_head	箭头的方向
IxOSD_ColorParams arrow_color	箭头的颜色
unsigned int reserved	保留字段

1.8.1.16 IxOSD_CircleParams

描述

(结构体) 在图像中绘制圆形时的参数

数据字段

unsigned int xc	圆心坐标的 x 坐标
unsigned int yc	圆心坐标的 y 坐标
unsigned int radius	圆的半径
IxOSD_ColorParams circle_color	圆边框的颜色
unsigned int has_bg_color	是否对圆进行填充的标志
IxOSD_ColorParams bg_color	圆的填充色
unsigned int reserved	保留字段

1.8.1.17 IxBaseMeta

描述

(结构体) 基础元数据

数据字段

struct _IxBatchMeta *batch_meta	batch 元数据
IxMetaType meta_type	元数据类型
void *uContext	上下文
IxMetaCopyFunc copy_func	元数据拷贝函数

IxMetaReleaseFunc release_func	元数据释放函数
--------------------------------	---------

1.8.1.18 IxUserMeta

描述

(结构体) 用户元数据

数据字段

IxBaseMeta base_meta	基础元数据
gpointer user_meta_data	用户元数据

1.8.1.19 IxLabelInfo

描述

(结构体) 标签的元数据

数据字段

IxBaseMeta base_meta	基础元数据
guint num_classes	给定标签的数量
gchar result_label[MAX_LABEL_SIZE]	标签数据
gchar *pResult_label	标签数据 (label 数据超过 MAX_LABEL_SIZE 大小后保存在这)
guint result_class_id	标签的 ID
guint label_id	多个分类器时保存标签 ID
gfloat result_prob	分类结果的可能性

1.8.1.20 IxClassifierMeta

描述

(结构体) 分类器的元数据

数据字段

IxBaseMeta base_meta	基础元数据
guint num_labels	目标的 label 数量

gint unique_component_id	分类器元数据的 ID
IxLabelInfoList *label_info_list	标签数据列表
const gchar *classifier_type	分类器的类型

1.8.1.21 IxDisplayMeta

描述

(结构体) 显示的元数据 (用户需要在图像上绘制时, 可以通过该结构体添加参数)

数据字段

IxBaseMeta base_meta	基础元数据
guint num_rects	矩形框的数量
guint num_labels	文本的数量
guint num_lines	线条的数量
guint num_arrows	箭头的数量
guint num_circles	圆的数量
guint num_polygons	多边形的数量
IxOSD_RectParams rect_params[MAX_ELEMENTS_IN_DISPLAY_META]	矩形框的参数信息
IxOSD_TextParams text_params[MAX_ELEMENTS_IN_DISPLAY_META]	文本的参数信息
IxOSD_LineParams line_params[MAX_ELEMENTS_IN_DISPLAY_META]	线条的参数信息
IxOSD_ArrowParams arrow_params[MAX_ELEMENTS_IN_DISPLAY_META]	箭头的参数信息
IxOSD_CircleParams circle_params[MAX_ELEMENTS_IN_DISPLAY_META]	圆的参数信息
IxOSD_PolygonParams polygon_params[MAX_ELEMENTS_IN_DISPLAY_META]	多边形的参数信息
gint64 reserved[MAX_RESERVED_FIELDS]	保留字段

1.8.1.22 IxObjectMeta

描述

(结构体) 目标的元数据

数据字段

IxBaseMeta base_meta	基础元数据
struct _IxObjectMeta *parent	目标元数据的父节点
gint unique_component_id	目标元数据的 ID
gint class_id	目标的类别 ID
guint64 object_id	目标经过跟踪后的 ID
Ix_BboxInfo detector_bbox_info	目标的检测框信息
Ix_BboxInfo tracker_bbox_info	目标经过跟踪后的检测框信息
gfloat confidence	目标的置信度
IxOSD_RectParams rect_params	目标的边框参数
IxOSD_MaskParams mask_params	目标的 mask 参数
IxOSD_TextParams text_params	目标的文本参数
gchar obj_label[MAX_LABEL_SIZE]	目标的标签信息
IxClassifierMetaList *classifier_meta_list	目标元数据中的分类器元数据列表
IxUserMetaList *obj_user_meta_list	目标元数据中的用户元数据列表
gint64 reserved[MAX_RESERVED_FIELDS]	保留字段

1.8.1.23 IxFrameMeta

描述

(结构体) 帧的元数据

数据字段

IxBaseMeta base_meta	基础元数据
guint pad_index	当前帧在 batch 中的编号
guint batch_id	当前帧所在 batch 的编号
guint frame_num	当前帧在数据流中的编号
guint64 buf_pts	当前帧的 pts 时间戳
guint64 ntp_timestamp	当前帧的 ntp 时间戳
guint source_id	当前帧在 batch 中的编号
source_frame_width	输入数据的宽度
source_frame_height	输入数据的高度
guint num_obj_meta	目标元数据的数量

gboolean bInferDone	帧数据是否推理的标志位
IxObjectMetaList *obj_meta_list	帧内目标的元数据列表
IxDisplayMetaList *display_meta_list	帧内部的显示元数据列表
IxUserMetaList *frame_user_meta_list	帧内部的用户元数据列表
guint pipeline_width	多路合并后输出的图像宽度
guint pipeline_height	多路合并后输出的图像高度
gint64 reserved[MAX_RESERVED_FIELDS]	保留字段

1.8.1.24 IxBatchMeta

描述

(结构体) batch 元数据

数据字段

IxBaseMeta base_meta	基础元数据
guint max_frames_in_batch	batch 中最大的帧数
guint num_frames_in_batch	batch 中的帧数
IxMetaPool *frame_meta_pool	帧元数据的内存池
IxMetaPool *obj_meta_pool	目标元数据的内存池
IxMetaPool *classifier_meta_pool	分类器元数据的内存池
IxMetaPool *display_meta_pool	显示元数据的内存池
IxMetaPool *user_meta_pool	用户元数据的内存池
IxMetaPool *label_info_meta_pool	标签元数据的内存池
IxFrameMetaList *frame_meta_list	帧元数据的列表
IxUserMetaList *batch_user_meta_list	batch 中用户元数据的列表
GRecMutex meta_mutex	元数据锁

1.8.1.25 IxInferSegmentationMeta

描述

(结构体) 分割网络输出元数据

数据字段

guint classes	分割网络输出的类别个数
guint width	分割网络输出类别图的宽度
guint height	分割网络输出类别图的高度
gint*class_map	分割网络输出类别图
gfloat *class_probabilities_map	分割网络输出的每种类别的概率数据
void *priv_data	私有数据
gint unique_id	元数据的唯一标识符

1.8.1.26 IxAnalyticsObjInfo

描述

(结构体) 分析插件输出的目标信息

数据字段

std::vector <std::string> roiStatus	保存目标在 ROI 区域的标签信息
std::vector <std::string> ocStatus	保存目标在 ROI 区域的过度拥挤标签信息
std::vector <std::string> lcStatus	保存目标的越线信息
std::string dirStatus	保存目标的方向信息
guint unique_id	元数据的唯一标识符
std::string objStatus	保留目标的状态标签信息

1.8.1.27 IxAnalyticsFrameMeta

描述

(结构体) 分析插件输出的帧的元数据

数据字段

std::unordered_map<std::string, bool> ocStatus	当前帧分析插件输出的过度拥挤状态信息
std::unordered_map<std::string, uint32_t> objInROICnt	保存当前帧在 ROI 区域的目标数量
std::unordered_map<std::string, uint64_t> objLCCurrCnt	保存当前帧越线目标数量和标签

std::unordered_map<std::string, uint64_t> objLCCumCnt	保存累积到当前帧的越线目标总数和标签
uint unique_id	元数据的唯一标识符
std::unordered_map<int, uint32_t> objCnt	当前帧每种类别目标的总数

1.8.1.28 IxInferDims

描述

(结构体) 推理网络的尺寸信息

数据字段

uint numDims	网络层的维度数量
unsigned int d[IXINFER_MAX_DIMS]	网络层每一个维度的尺寸
unsigned int numElements	网络层所有维度的尺寸和

1.8.1.29 IxInferDimsCHW

描述

(结构体) 推理网络的尺寸信息 (数据排布为 CHW)

数据字段

uint c	网络层的通道数量
uint h	网络层的高度
uint w	网络层的宽度

1.8.1.30 IxInferDataType

描述

(结构体) 网络层的数据类型

数据字段

FLOAT	0 浮点型
HALF	1 半精度浮点

INT8	2 字符型
INT32	3 整型

1.8.1.31 IxInferLayerInfo

描述

(结构体) 推理网络的层信息

数据字段

IxInferDataType dataType	网络层的数据类型
IxInferDims inferDims	网络层的维度信息
int bindingIndex	网络层的索引
const char *layerName	网络层的名称
void *buffer	网络层的数据
int isInput	当前层是否为输入层的标志

1.8.1.32 IxInferNetworkInfo

描述

(结构体) 推理网络的信息

数据字段

guint width	网络输入的宽度
guint height	网络层的高度
guint channels	网络层的通道数量

1.8.1.33 IxInferObjectDetectionInfo

描述

(结构体) 检测目标的相关信息

数据字段

guint classId	目标的类别 ID
---------------	----------

gfloat left	目标检测框左上顶点在水平方向的偏移
gfloat top	目标检测框左上顶点在垂直方向的偏移
gfloat width	目标检测框的宽度
gfloat height	目标检测框的高度
float detectionConfidence	目标的置信度

1.8.1.34 IxInferAttribute

描述

(结构体) 分类的属性信息

数据字段

unsigned int attributeIndex	保存属性标签的索引
unsigned int attributeValue	保存属性的输出值
float attributeConfidence	保存属性的置信度
char *attributeLabel	保存属性标签值

1.8.1.35 IxInferParseDetectionParams

描述

(结构体) 需要解析的检测参数

数据字段

guint numClassesConfigured	类别的数量
std::vector<gfloat> perClassPreclusterThreshold	保存聚类时使用的每一个类别的阈值
std::vector<gfloat> perClassPostclusterThreshold	保存聚类后使用的每一个类别的阈值
std::vector<gfloat> &perClassThreshold	保存属性标签值

1.8.2 函数接口

1.8.2.1 ix_acquire_frame_meta_from_pool

功能概述

从内存池中获取帧的元数据的内存

函数原型

```
IxFrameMeta *ix_acquire_frame_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxFrameMeta 类型的指针

1.8.2.2 ix_add_frame_meta_to_batch

功能概述

将帧的元数据添加到 batch 的元数据中

函数原型

```
void ix_add_frame_meta_to_batch(IxBatchMeta *batch_meta, IxFrameMeta *frame_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxFrameMeta *frame_meta: 需要添加的帧的元数据指针

返回值

无

1.8.2.3 ix_remove_frame_meta_from_batch

功能概述

从 batch 的元数据中移除帧的元数据

函数原型

```
void ix_remove_frame_meta_from_batch(IxBatchMeta *batch_meta, IxFrameMeta *frame_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxFrameMeta *frame_meta: 需要移除的帧的元数据指针

返回值

无

1.8.2.4 ix_acquire_obj_meta_from_pool

功能概述

从内存池中获取目标的元数据内存

函数原型

```
IxObjectMeta *ix_acquire_obj_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxObjectMeta 类型的指针

1.8.2.5 ix_add_obj_meta_to_frame

功能概述

将目标的元数据添加到帧的元数据中

函数原型

```
void ix_add_obj_meta_to_frame(IxFrameMeta *frame_meta, IxObjectMeta *obj_meta,  
↪ IxObjectMeta *obj_parent);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxObjectMeta *obj_meta: 需要添加的目标元数据指针
- IxObjectMeta *obj_parent: 目标元数据的父节点, 可以为空

返回值

无

1.8.2.6 ix_remove_obj_meta_from_frame

功能概述

从帧的元数据中移除目标的元数据

函数原型

```
void ix_remove_obj_meta_from_frame(IxFrameMeta *frame_meta, IxObjectMeta *obj_meta);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxObjectMeta *obj_meta: 需要移除的目标元数据

返回值

无

1.8.2.7 ix_acquire_classifier_meta_from_pool

功能概述

从内存池中获取分类器的元数据内存

函数原型

```
IxClassifierMeta *ix_acquire_classifier_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxClassifierMeta 类型的指针

1.8.2.8 ix_add_classifier_meta_to_obj

功能概述

将分类器的元数据添加到目标的元数据中

函数原型

```
void ix_add_classifier_meta_to_obj(IxObjectMeta *obj_meta, IxClassifierMeta  
↪ *classifier_meta);
```

参数说明

- IxObjectMeta *obj_meta: 目标的元数据指针
- IxClassifierMeta *classifier_meta: 需要添加的分类器元数据指针

返回值

无

1.8.2.9 ix_remove_classifier_meta_from_obj

功能概述

从目标的元数据中移除分类器的元数据

函数原型

```
void ix_remove_classifier_meta_from_obj(IxObjectMeta *obj_meta, IxClassifierMeta  
↪ *classifier_meta);
```

参数说明

- IxObjectMeta *obj_meta: 目标的元数据指针
- IxClassifierMeta *classifier_meta: 需要移除的分类器元数据

返回值

无

1.8.2.10 ix_acquire_display_meta_from_pool

功能概述

从内存池中获取显示元数据的内存

函数原型

```
IxDisplayMeta *ix_acquire_display_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxDisplayMeta 类型的指针

1.8.2.11 ix_add_display_meta_to_frame

功能概述

将需要显示的元数据添加到帧的元数据中

函数原型

```
void ix_add_display_meta_to_frame(IxFrameMeta *frame_meta, IxDisplayMeta *display_meta);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxDisplayMeta *display_meta: 需要添加的显示元数据

返回值

无

1.8.2.12 ix_remove_display_meta_from_frame

功能概述

从帧的元数据中移除要显示的元数据

函数原型

```
void ix_remove_display_meta_from_frame(IxFrameMeta *frame_meta, IxDisplayMeta  
↪ *display_meta);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxDisplayMeta *display_meta: 需要移除的显示元数据

返回值

无

1.8.2.13 ix_acquire_label_info_meta_from_pool

功能概述

从内存池中获取标签元数据的内存

函数原型

```
IxLabelInfo *ix_acquire_label_info_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxLabelInfo 类型的指针

1.8.2.14 ix_add_label_info_meta_to_classifier

功能概述

将标签数据添加到分类器的元数据中

函数原型

```
void ix_add_label_info_meta_to_classifier(IxClassifierMeta *classifier_meta, IxLabelInfo  
↪ *label_info_meta);
```

参数说明

- IxClassifierMeta *classifier_meta: 分类器的元数据指针

- IxLabelInfo *label_info_meta: 需要添加的标签数据

返回值

无

1.8.2.15 ix_remove_label_info_meta_from_classifier

功能概述

从分类器的元数据中移除标签元数据

函数原型

```
void ix_remove_label_info_meta_from_classifier(IxClassifierMeta *classifier_meta,  
↪ IxLabelInfo *label_info_meta);
```

参数说明

- IxClassifierMeta *classifier_meta: 分类器的元数据指针
- IxLabelInfo *label_info_meta: 需要移除的标签元数据

返回值

无

1.8.2.16 ix_acquire_user_meta_from_pool

功能概述

从内存池中获取用户元数据的内存

函数原型

```
IxUserMeta *ix_acquire_user_meta_from_pool(IxBatchMeta *batch_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针

返回值

- IxUserMeta 类型的指针

1.8.2.17 ix_add_user_meta_to_batch

功能概述

将用户元数据添加到 batch 的元数据中

函数原型

```
void ix_add_user_meta_to_batch(IxBatchMeta *batch_meta, IxUserMeta *user_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxUserMeta *user_meta: 需要添加的用户元数据指针

返回值

无

1.8.2.18 ix_remove_user_meta_from_batch

功能概述

从 batch 的元数据中移除用户元数据

函数原型

```
void ix_remove_user_meta_from_batch(IxBatchMeta *batch_meta, IxUserMeta *user_meta);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxUserMeta *user_meta: 需要移除的用户元数据指针

返回值

无

1.8.2.19 ix_add_user_meta_to_frame

功能概述

将用户元数据添加到帧的元数据中

函数原型

```
void ix_add_user_meta_to_frame(IxFrameMeta *frame_meta, IxUserMeta *user_meta);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxUserMeta *user_meta: 需要添加的用户元数据指针

返回值

无

1.8.2.20 ix_remove_user_meta_from_frame

功能概述

从帧的元数据中移除用户元数据

函数原型

```
void ix_remove_user_meta_from_frame(IxFrameMeta *frame_meta, IxUserMeta *user_meta);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxUserMeta *user_meta: 需要移除的用户元数据

返回值

无

1.8.2.21 ix_add_user_meta_to_obj

功能概述

将用户元数据添加到目标的元数据中

函数原型

```
void ix_add_user_meta_to_obj(IxObjectMeta *obj_meta, IxUserMeta *user_meta);
```

参数说明

- IxObjectMeta *obj_meta: 目标的元数据指针
- IxUserMeta *user_meta: 需要添加的用户元数据指针

返回值

无

1.8.2.22 ix_remove_user_meta_from_obj

功能概述

从目标的元数据内存中移除用户元数据

函数原型

```
void ix_remove_user_meta_from_obj(IxObjectMeta *obj_meta, IxUserMeta *user_meta);
```

参数说明

- IxObjectMeta *obj_meta: 目标的元数据指针
- IxUserMeta *user_meta: 需要移除的用户元数据

返回值

无

1.8.2.23 ix_get_nth_frame_meta

功能概述

从帧的元数据列表中获取第 n 帧的元数据

函数原型

```
IxFrameMeta * ix_get_nth_frame_meta(IxFrameMetaList *frame_meta_list, guint index);
```

参数说明

- IxFrameMetaList *frame_meta_list: 帧的元数据列表指针
- guint index: 需要获取的元数据的序号

返回值

- IxFrameMeta 类型的指针

1.8.2.24 ix_clear_frame_meta_list

功能概述

从 batch 的元数据中删除帧的元数据列表

函数原型

```
void ix_clear_frame_meta_list(IxBatchMeta *batch_meta, IxFrameMetaList *meta_list);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxFrameMetaList *meta_list: 需要删除的帧的元数据列表

返回值

无

1.8.2.25 ix_clear_obj_meta_list

功能概述

从帧的元数据中删除目标的元数据列表

函数原型

```
void ix_clear_obj_meta_list(IxFrameMeta *frame_meta, IxObjectMetaList *meta_list);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxObjectMetaList *meta_list: 需要删除的目标元数据列表

返回值

无

1.8.2.26 ix_clear_classifier_meta_list

功能概述

从目标元数据中删除分类器的元数据列表

函数原型

```
void ix_clear_classifier_meta_list(IxObjectMeta *obj_meta, IxClassifierMetaList  
↪ *meta_list);
```

参数说明

- IxObjectMeta *obj_meta: 目标的元数据指针
- IxClassifierMetaList *meta_list: 需要删除的分类器元数据列表

返回值

无

1.8.2.27 ix_clear_label_info_meta_list

功能概述

从分类器元数据中删除标签列表

函数原型

```
void ix_clear_label_info_meta_list(IxClassifierMeta *classifier_meta, IxLabelInfoList  
↪ *meta_list);
```

参数说明

- IxClassifierMeta *classifier_meta: 分类器的元数据指针
- IxLabelInfoList *meta_list: 需要删除的标签列表

返回值

无

1.8.2.28 ix_clear_display_meta_list

功能概述

从帧的元数据中删除显示元数据

函数原型

```
void ix_clear_display_meta_list(IxFrameMeta *frame_meta, IxDisplayMetaList *meta_list);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxDisplayMetaList *meta_list: 需要删除的显示元数据列表

返回值

无

1.8.2.29 ix_clear_batch_user_meta_list

功能概述

从 batch 的元数据中删除用户元数据列表

函数原型

```
void ix_clear_batch_user_meta_list(IxBatchMeta *batch_meta, IxUserMetaList *meta_list);
```

参数说明

- IxBatchMeta *batch_meta: batch 的元数据指针
- IxUserMetaList *meta_list: 需要删除的用户元数据列表

返回值

无

1.8.2.30 ix_clear_frame_user_meta_list

功能概述

从帧的元数据中删除用户元数据列表

函数原型

```
void ix_clear_frame_user_meta_list(IxFrameMeta *frame_meta, IxUserMetaList *meta_list);
```

参数说明

- IxFrameMeta *frame_meta: 帧的元数据指针
- IxUserMetaList *meta_list: 需要删除的用户元数据列表

返回值

无

1.8.2.31 ix_get_user_meta_type

功能概述

获取用户元数据的类型

函数原型

```
IxMetaType ix_get_user_meta_type(gchar *meta_descriptor);
```

参数说明

- gchar *meta_descriptor: 用户元数据的字符串描述符

返回值

- IxMetaType 的枚举值

1.8.2.32 gst_buffer_add_ix_meta

功能概述

在内存中添加元数据

函数原型

```
IxMeta *gst_buffer_add_ix_meta(GstBuffer *buffer, gpointer meta_data, gpointer user_data,
    IxMetaCopyFunc copy_func, IxMetaReleaseFunc release_func);
```

参数说明

- GstBuffer *buffer: 需要添加元数据的内存指针
- gpointer meta_data: 需要添加的元数据
- gpointer user_data: 需要添加的用户数据
- IxMetaCopyFunc copy_func: 用来拷贝 meta 数据的拷贝函数
- IxMetaReleaseFunc release_func: 用来释放 meta 数据的函数

返回值

- IxMeta 类型的指针

1.8.2.33 gst_buffer_get_ix_meta

功能概述

从内存中获取元数据

函数原型

```
IxMeta *gst_buffer_get_ix_meta(GstBuffer *buffer);
```

参数说明

- GstBuffer *buffer: 获取元数据的内存指针

返回值

- IxMeta 类型的指针

1.8.2.34 gst_buffer_get_ix_batch_meta

功能概述

从内存中获取 batch 的元数据指针

函数原型

```
IxBatchMeta *gst_buffer_get_ix_batch_meta(GstBuffer *buffer);
```

参数说明

- GstBuffer *buffer: 获取元数据的内存指针

返回值

- IxBatchMeta 类型的指针

1.8.2.35 IxInferParseCustomFunc

功能概述

定义检测网络的后处理函数指针

函数原型

```
typedef bool (*IxInferParseCustomFunc)(std::vector<IxInferLayerInfo> const
↳ &outputLayersInfo, IxInferNetworkInfo const &networkInfo, IxInferParseDetectionParams
↳ const &detectionParams, std::vector<IxInferObjectDetectionInfo> &objectList);
```

参数说明

- std::vector<IxInferLayerInfo> const &outputLayersInfo: 存储推理中网络输出层的信息
- IxInferNetworkInfo const &networkInfo: 存储模型的信息
- IxInferParseDetectionParams const &detectionParams: 存储检测网络的相关参数
- std::vector<IxInferObjectDetectionInfo> &objectList: 存储检测完后目标的相关信息

返回值

无

1.8.2.36 IxInferClassifierParseCustomFunc

功能概述

定义分类网络的后处理函数指针

函数原型

```
typedef bool (*IxInferClassifierParseCustomFunc)(std::vector<IxInferLayerInfo> const
↳ &outputLayersInfo, IxInferNetworkInfo const &networkInfo, float classifierThreshold,
↳ std::vector<IxInferAttribute> &attrList, std::string &descString);
```

参数说明

- `std::vector<IxInferLayerInfo> const &outputLayersInfo`: 存储推理中网络输出层的信息
- `IxInferNetworkInfo const &networkInfo`: 存储模型的信息
- `float classifierThreshold`: 存储分类的阈值
- `std::vector<IxInferAttribute> &attrList`: 存储分类网络的属性信息
- `std::string &descString`: 描述字符串

返回值

无

1.8.2.37 IxInferSemSegmentationParseCustomFunc

功能概述

定义分割网络的后处理函数指针

函数原型

```
typedef bool (* IxInferSemSegmentationParseCustomFunc) (std::vector<IxInferLayerInfo>
↪ const &outputLayersInfo, IxInferNetworkInfo const &networkInfo, float
↪ segmentationThreshold, unsigned int numClasses, int* classificationMap, float
↪ *classProbabilityMap);
```

参数说明

- `std::vector<IxInferLayerInfo> const &outputLayersInfo`: 存储推理中网络输出层的信息
- `IxInferNetworkInfo const &networkInfo`: 存储模型的信息
- `float segmentationThreshold`: 存储分割的阈值
- `unsigned int numClasses`: 分割的类别
- `int* classificationMap`: 存储分割网络的输出类别图
- `float *classProbabilityMap`: 存储网络输出每种类别的概率

返回值

无

1.8.2.38 ix_parse_file_source

功能概述

从 json 参数文件中给 filesrc 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_file_source (GstElement *element, gchar *cfg_file_path, const
↪ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 filesrc 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.39 ix_parse_rtsp_source

功能概述

从 json 参数文件中给 rtspsrc 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_rtsp_source (GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 rtspsrc 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.40 ix_parse_vpu_decode

功能概述

从 json 参数文件中给Gst-IxVpuDecode 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_vpu_decode (GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 要设置参数的Gst-IxVpuDecode 插件指针

- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.41 ix_parse_streammux

功能概述

从 json 参数文件中给 **Gst-IxStreamMux** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_streammux(GstElement *element, gchar *cfg_file_path, const  
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxStreamMux** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.42 ix_parse_tracker

功能概述

从 json 参数文件中给 **Gst-IxTracker** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_tracker(GstElement *element, gchar *cfg_file_path, const char  
↳ *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxTracker** 插件指针
- gchar *cfg_file_path: json 文件的路径

- `const char *group`: 插件所在的组

返回值

`IxJsonParserStatus`:

- `IX_JSON_PARSER_SUCCESS`: 解析成功
- `IX_JSON_PARSER_DISABLED`: 属性设置未生效
- `IX_JSON_PARSER_ERROR`: 解析失败

1.8.2.43 ix_parse_osd

功能概述

从 json 参数文件中给 `Gst-IxOsd` 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_osd(GstElement *element, gchar *cfg_file_path, const char
↪ *group);
```

参数说明

- `GstElement *element`: 需要设置参数的 `Gst-IxOsd` 插件指针
- `gchar *cfg_file_path`: json 文件的路径
- `const char *group`: 插件所在的组

返回值

`IxJsonParserStatus`:

- `IX_JSON_PARSER_SUCCESS`: 解析成功
- `IX_JSON_PARSER_DISABLED`: 属性设置未生效
- `IX_JSON_PARSER_ERROR`: 解析失败

1.8.2.44 ix_parse_tiler

功能概述

从 json 参数文件中给 `Gst-IxMultiStreamTiler` 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_tiler(GstElement *element, gchar *cfg_file_path, const char
↪ *group);
```

参数说明

- `GstElement *element`: 需要设置参数的 `Gst-IxMultiStreamTiler` 插件指针
- `gchar *cfg_file_path`: json 文件的路径
- `const char *group`: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.45 ix_parse_gie

功能概述

从 json 参数文件中给Gst-IxInfer 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_gie(GstElement *element, gchar *cfg_file_path, const char  
↪ *group);
```

参数说明

- GstElement *element: 需要设置参数的Gst-IxInfer 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.46 ix_parse_file_sink

功能概述

从 json 参数文件中给 filesink 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_file_sink(GstElement *element, gchar *cfg_file_path, const  
↪ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 filesink 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.47 ix_parse_fake_sink

功能概述

从 json 参数文件中给 fakesink 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_fake_sink(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 fakesink 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.48 ix_parse_h264_encode

功能概述

从 json 参数文件中给 Gst-IxH264Encode 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_h264_encode(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 Gst-IxH264Encode 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.49 ix_parse_jpu_encode

功能概述

从 json 参数文件中给 **Gst-IxJpuEncode** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_jpu_encode(GstElement *element, gchar *cfg_file_path, const  
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxJpuEncode** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.50 ix_parse_analytics

功能概述

从 json 参数文件中给 **Gst-IxAnalytics** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_analytics(GstElement *element, gchar *cfg_file_path, const  
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxAnalytics** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.51 ix_parse_seg_visual

功能概述

从 json 参数文件中给 **Gst-IxSegVisual** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_seg_visual(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxSegVisual** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.52 ix_parse_converter

功能概述

从 json 参数文件中给 **Gst-IxConverter** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_converter(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxConverter** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.53 ix_parse_preprocess

功能概述

从 json 参数文件中给 **Gst-IxPreProcess** 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_preprocess(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的 **Gst-IxPreProcess** 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.54 ix_get_json_list

功能概述

(用于多流的情况) 从 json 文件中获取某个插件的参数列表

函数原型

```
IxJsonParserStatus ix_get_json_list(GList **para_list, gchar *cfg_file_path, const char
↳ *group);
```

参数说明

- GList **para_list: 返回的插件参数列表
- gchar *cfg_file_path: json 文件的路径
- const char *group: 需要获取参数的插件组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.55 ix_parse_file_source_list

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给 filesrc 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_file_source_list (GstElement *element, gpointer para_data);
```

参数说明

- GstElement *element: 需要设置参数的 filesrc 插件指针
- gpointer para_data: 参数列表

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.56 ix_parse_jpu_encode_list

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给Gst-IxJpuEncode 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_jpu_encode_list (GstElement *element, gpointer para_data);
```

参数说明

- GstElement *element: 需要设置参数的Gst-IxJpuEncode 插件指针
- gpointer para_data: 参数列表

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.57 ix_parse_file_sink_list

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给 filesink 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_file_sink_list(GstElement *element, gpointer para_data);
```

参数说明

- GstElement *element: 需要设置参数的 filesink 插件指针
- gpointer para_data: 参数列表

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.58 ix_parse_h264_encode_list

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给Gst-IxH264Encode 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_h264_encode_list (GstElement *element, gpointer para_data);
```

参数说明

- GstElement *element: 需要设置参数的Gst-IxH264Encode 插件指针
- gpointer para_data: 参数列表

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.59 ix_parse_rtsp_source_list

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给 rtspsrc 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_rtsp_source_list (GstElement *element, gpointer para_data);
```

参数说明

- GstElement *element: 需要设置参数的 rtspsrc 插件指针

- gpointer para_data: 参数列表

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.60 ix_parse_jpu_decode

功能概述

(用于多流的情况) 用从ix_get_json_list 接口获取到的参数列表给Gst-IxJpuDecode 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_jpu_decode(GstElement *element, gchar *cfg_file_path, const
↳ char *group);
```

参数说明

- GstElement *element: 需要设置参数的Gst-IxJpuDecode 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.8.2.61 ix_parse_multi_file_source

功能概述

从 json 参数文件中给 multifilesrc 插件设置参数

函数原型

```
IxJsonParserStatus ix_parse_multi_file_source(GstElement *element, gchar *cfg_file_path,
↳ const char *group);
```

参数说明

- GstElement *element: 需要设置参数的 multifilesrc 插件指针
- gchar *cfg_file_path: json 文件的路径
- const char *group: 插件所在的组

返回值

IxJsonParserStatus:

- IX_JSON_PARSER_SUCCESS: 解析成功
- IX_JSON_PARSER_DISABLED: 属性设置未生效
- IX_JSON_PARSER_ERROR: 解析失败

1.9 常见问题

1.9.1 环境相关问题

问题 1: gi 版本不匹配

问题描述

在执行 **import gi** 时报以下错误:

```
ImportError: cannot import name '_gi' from 'gi' (/usr/lib/python3/dist-packages/
↳ gi/__init__.py)
```

解决方法 1

执行以下命令解决问题:

```
# x86 环境执行以下命令
$ sudo ln -s /usr/lib/python3/dist-packages/
↳ gi/_gi.cpython-{36m,37m}-x86_64-linux-gnu.so

# ARM 环境执行以下命令
$ sudo ln -s /usr/lib/python3/dist-packages/
↳ gi/_gi.cpython-{36m,37m}-aarch64-linux-gnu.so
```

解决方法 2

执行以下命令解决问题:

```
$ python3 -m pip install --ignore-installed PyGObject
```

问题 2: GStreamer 无法加载 Gst-IxInfer Plugin

问题描述

在执行 **gst-inspect-1.0** 时报以下错误:

```
(gst-plugin-scanner:2280): GStreamer-WARNING **: 06:27:46.886: Failed to load plugin
↳ '/opt/ixstream/plugins/libixstream_infer.so': libtvm.so: cannot open shared
↳ object file: No such file or directory
```

解决方法

参考以下命令将 libtvm.so 的路径添加到 LD_LIBRARY_PATH 变量中:

```
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib/python3.7/dist-packages/tvm/
↪  cpp
```

问题 3: PyGObject 安装失败

问题描述

在执行 **pip3 install PyGObject** 时报以下错误:

```
Requested 'glib-2.0 >=2.64.0' but version of Glib is 2.56.1
```

解决方法

安装 PyGObject 时指定版本安装, 相关命令如下:

```
$ pip3 install PyGObject==3.44.0
```

问题 4: jpegparse 解析报错

问题描述

在使用 jpegparse 解析 sample 中的 mjpeg.mp4 视频时报以下错误:

```
(gst-launch-1.0:64681): CRITICAL **: 06:35:52.087: _masked_scan_uint32_peek:
↪ assertion '(guint64) offset + size <= reader->size - reader->byte' failed
```

解决方法

更新 gst 版本, 不影响结果时可忽略, 相关命令如下:

```
$ apt upgrade gstreamer1.0 gstreamer1.0-plugins-base gstreamer1.0-plugins-good
↪ gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly gstreamer1.0-libav
↪ gstreamer1.0-doc gstreamer1.0-tools gstreamer1.0-x gstreamer1.0-alsa
↪ gstreamer1.0-gl gstreamer1.0-gtk3
```

问题 5: qtdemux 在某些视频上报错

问题描述

在 pipeline 中读取某些视频时会报以下错误:

```
ERROR: from element /GstPipeline:pipeline0/GstQTDemux:qtdemux0: Internal data stream
↪ error.
Additional debug info:
qtdemux.c(6619): gst_qtdemux_loop (): /GstPipeline:pipeline0/GstQTDemux:qtdemux0:
streaming stopped, reason not-negotiated (-4)
```

解决方法

出现该问题的原因是因为输入视频是采用较高的 bit depth 做编码, 而 IxStream 中目前仅支持普通的 8bit, 可以通过 ffmpeg 命令验证视频是否为高 bit 位深, 命令如下:

```
$ ffmpeg -i input.mp4
```

如果确定视频为高 bit depth，可以在 pipeline 中加入 videoconvert 插件，做类型转换，相关命令如下，可参照修改：

```
# 错误命令
$ gst-launch-1.0 filesrc location="input.mp4" blocksize=131072 ! atdemux ! h264parse
  ↪ ! avdec h264 ! Gst-IxConverter ! "video/x-raw(memory:IXMM)" ! Gst-IxStreamMux
  ↪ width=1280 height=720 live source=false ! Gst-IxConverter ! video/x-raw !
  ↪ ximagesink

# 正确命令 (在原命令基础上增加 videoconvert)
$ gst-launch-1.0 filesrc location="input.mp4" blocksize=131072 ! qtdemux ! h264parse
  ↪ ! avdec_h264 ! videoconvert ! Gst-IxConverter ! "video/x-raw(memory:IXMM)" !
  ↪ Gst-IxStreamMux width=1280 height=720 live_source=false ! Gst-IxConverter !
  ↪ video/x-raw ! ximagesink
```

1.10 IxStream 与 DeepStream 配置接口对比一览表

表 90: 配置接口对比

IxStream	DeepStream	说明
Gst-IxVpuDecode	Gst-nvvideo4linux2(decoder)	
Gst-IxH264Encode	Gst-nvvideo4linux2(encoder)	Gst-nvvideo4linux2 使用专用的硬件加速，Gst-IxH264Encode 使用 CUDA Core 实现
Gst-IxJpuDecode	Gst-nvjpegdec	
Gst-IxJpuEncode	Gst-nvjpegenc	
Gst-IxConverter	Gst-nvvideoconvert	
Gst-IxStreamMux	Gst-nvstreammux	
Gst-IxStreamDemux	Gst-nvstreamdemux	
Gst-IxPreprocess	Gst-nvdspreprocess	Alpha 版本，功能尚未完善
Gst-IxInfer	Gst-nvinfer	
Gst-IxTracker	Gst-nvtracker	
Gst-IxOsd	Gst-nvdsosd	
Gst-IxMultiStreamTiler	Gst-nvmultistreamtiler	

IxStream	DeepStream	说明
Gst-IxAnalytics	Gst-nvdsanalytics	
Gst-IxSegVisual	Gst-nvsegvisual	

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。